

**LAPORAN AKHIR PRAKTIKUM
PEMROGRAMAN MOBILE**



Oleh:

Regina Silva Maharatini

NIM. 2310817220011

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2025**

LEMBAR PENGESAHAN
LAPORAN AKHIR PRAKTIKUM PEMROGRAMAN WEB II

Laporan Akhir Praktikum Pemrograman Mobile ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Akhir Praktikum ini dikerjakan oleh:

Nama Praktikan : Regina Silva Maharatini

NIM : 2310817220011

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Zulfa Auliya Akbar
NIM. 2210817210026

Muti`a Maulida S.Kom M.T.I
NIP. 19881027 201903 20 13

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR GAMBAR	4
DAFTAR TABEL	5
MODUL 1 : Android Basic in Kotlin	6
SOAL 1.....	6
A.Source Code.....	8
B.Output Program	14
C.Pembahasan	14
MODUL 2 : Android Layout	17
SOAL 1.....	17
A. Source Code.....	19
B. Output Program	27
C. Pembahasan	27
MODUL 3 : Build a Scrollable List.....	29
SOAL 1	29
A. Source Code.....	31
B. Output Program	46
C. Pembahasan	46
MODUL 4 : ViewModel and Debugging	51
SOAL 1	51
A. Source Code.....	52
B. Output Program	68
C. Pembahasan	70
MODUL 5 : Connect to the Internet	74
SOAL 1	74
A. Source Code.....	75
B. Output Program	93
C. Pembahasan	94
Tautan Git.....	99

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1 Modul 1	14
Gambar 2. Screenshot Hasil Jawaban Soal 1 Modul 2	27
Gambar 3 . Screenshot Hasil Jawaban Soal 1 Modul 3	46
Gambar 4. Screenshot Tampilan Aplikasi	68
Gambar 5. Screenshot Hasil Logging	68
Gambar 6. Screenshot Penggunaan Debugger	69
Gambar 7. Screenshot Hasil Jawaban Soal 1 Modul 5	93

DAFTAR TABEL

Tabel 1. Source Code Soal 1 Modul 1	8
Tabel 2. Source Code Soal 1 Modul 1	11
Tabel 3. Source Code Soal 1 Modul 1	11
Tabel 4. Source Code Soal 1 Modul 2	19
Tabel 5. Source Code Soal 1 Modul 2	21
Tabel 6. Source Code Soal 1 Modul 2	25
Tabel 7. Source Code Soal 1 Modul 3	31
Tabel 8. Source Code Soal 1 Modul 3	39
Tabel 9. Source Code Soal 1 Modul 3	43
Tabel 10. Source Code Soal 1 Modul 4	52
Tabel 11. Source Code Soal 1 Modul 4	59
Tabel 12. Source Code Soal 1 Modul 4	62
Tabel 13. Source Code Soal 1 Modul 4	66
Tabel 14. Source Code Soal 1 Modul 4	67
Tabel 15. Source Code Soal 1 Modul 5	75
Tabel 16. Source Code Soal 1 Modul 5	75
Tabel 17. Source Code Soal 1 Modul 5	76
Tabel 18. Source Code Soal 1 Modul 5	77
Tabel 19. Source Code Soal 1 Modul 5	78
Tabel 20. Source Code Soal 1 Modul 5	82
Tabel 21. Source Code Soal 1 Modul 5	84
Tabel 22. Source Code Soal 1 Modul 5	88
Tabel 23. Source Code Soal 1 Modul 5	89
Tabel 24. Source Code Soal 1 Modul 5	91

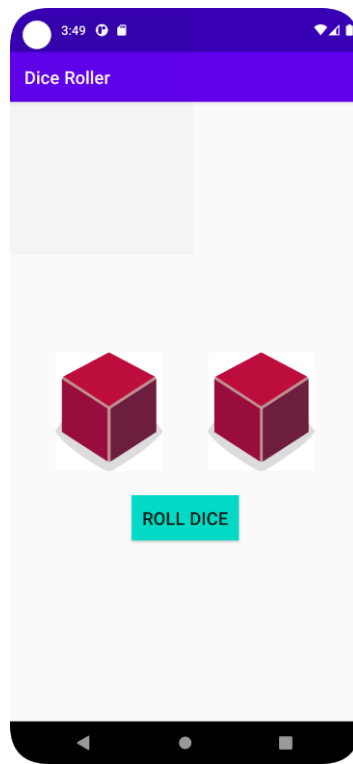
MODUL 1 : Android Basic in Kotlin

SOAL 1

Soal Praktikum:

Buatlah sebuah aplikasi yang dapat menampilkan 2 (dua) buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll Dice”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

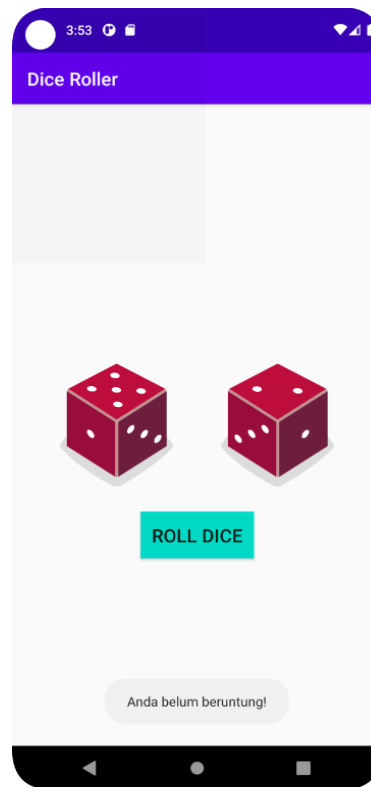
1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



Gambar 1 Tampilan Awal Aplikasi

2. Setelah user menekan tombol “Roll Dice” maka masing-masing dadu akan memunculkan sisi dadu masing-masing dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2 maka

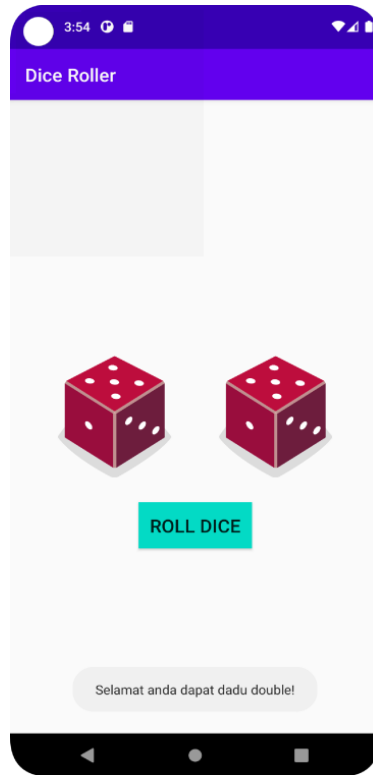
akan menampilkan pesan “Anda belum beruntung!” seperti dapat dilihat pada Gambar 2.



Gambar 2 Tampilan Dadu Setelah Di Roll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat anda dapat dadu double!” seperti dapat dilihat pada Gambar 3.
4. Upload aplikasi yang telah anda buat kedalam repository github ke dalam **folder Module 2 dalam bentuk project**. Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repo.

5. Untuk gambar dadu dapat didownload pada link berikut:
https://drive.google.com/u/0/uc?id=147HT2lIH5qin3z5ta7H9y2N_5OMW81Ll&export=download



Gambar 3 Tampilan Roll Dadu Double

A. Source Code

1. MainActivity.kt

Tabel 1. Source Code Soal 1 Modul 1

1	<code>package com.example.daduapp</code>
2	
3	<code>import android.os.Bundle</code>
4	<code>import android.widget.Toast</code>

5	import androidx.activity.viewModels
6	import androidx.appcompat.app.AppCompatActivity
7	import
8	com.example.daduapp.databinding.ActivityMainBinding
9	
10	class MainActivity : AppCompatActivity() {
11	
12	private lateinit var binding:
13	ActivityMainBinding
14	
15	private val viewModel: DiceViewModel by
16	viewModels()
17	
18	override fun onCreate(savedInstanceState:
19	Bundle?) {
20	super.onCreate(savedInstanceState)
21	
22	binding =
23	ActivityMainBinding.inflate(layoutInflater)
24	setContentView(binding.root)
25	
26	updateDiceImages(viewModel.dice1,
27	viewModel.dice2)
28	
29	binding.rollbuton.setOnClickListener {
30	viewModel.dice1 = Dice(6).roll()
31	viewModel.dice2 = Dice(6).roll()
32	
33	updateDiceImages(viewModel.dice1,
34	viewModel.dice2)

```

35         }
36     }
37
38     private fun updateDiceImages(dice1: Int, dice2:
39 Int) {
40
41 binding.diceImage1.setImageResource(getDiceDrawable(
42 dice1))
43
44 binding.diceImage2.setImageResource(getDiceDrawable(
45 dice2))
46
47         if (dice1 == dice2 && dice1 != 0) {
48             Toast.makeText(this, "Selamat anda dapat
49 dadu double!", Toast.LENGTH_SHORT).show()
50         } else if (dice1 != 0 && dice2 != 0) {
51             Toast.makeText(this, "Anda belum
52 beruntung!", Toast.LENGTH_SHORT).show()
53         }
54     }
55
56     private fun getDiceDrawable(roll: Int): Int {
57         return when (roll) {
58             1 -> R.drawable.dice_1
59             2 -> R.drawable.dice_2
60             3 -> R.drawable.dice_3
61             4 -> R.drawable.dice_4
62             5 -> R.drawable.dice_5
63             6 -> R.drawable.dice_6
64             else -> R.drawable.dice_0

```

65	}
66	}
67	}
68	
69	class Dice(private val numSides: Int) {
70	fun roll(): Int = (1..numSides).random()
71	}
72	

2. DaduView.kt

Tabel 2. Source Code Soal 1 Modul 1

1	package com.example.daduapp
2	
3	import androidx.lifecycle.ViewModel
4	
5	class DiceViewModel : ViewModel() {
6	var dice1: Int = 0
7	var dice2: Int = 0
8	}
9	
10	

3. activity_main.xml

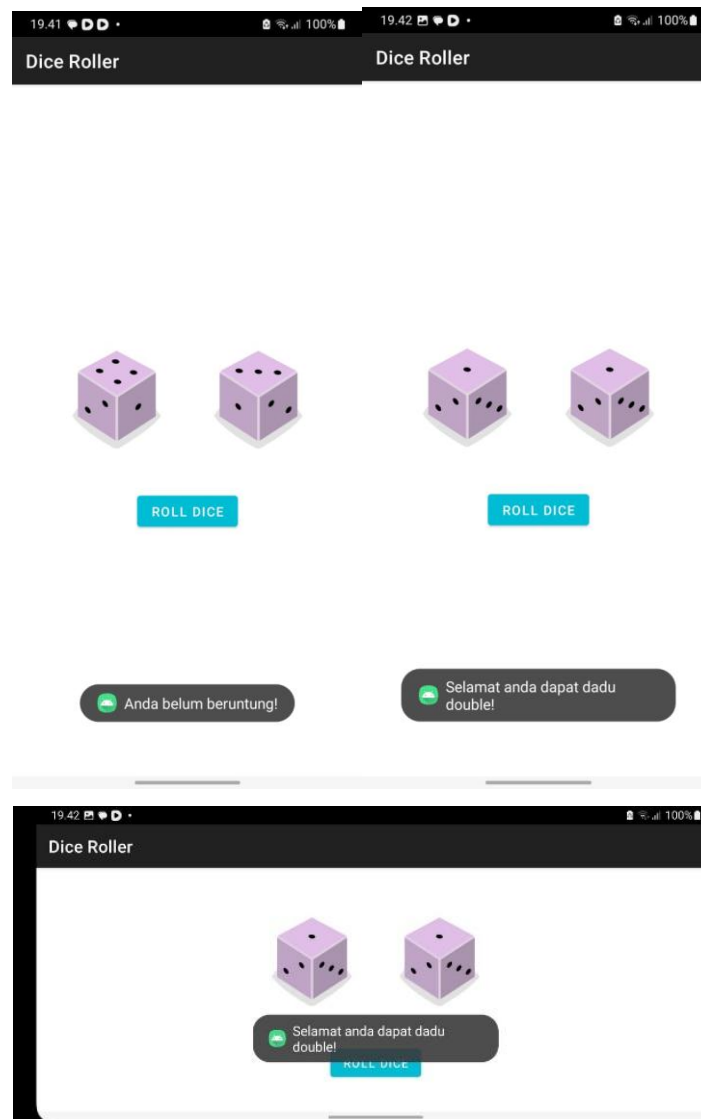
Tabel 3. Source Code Soal 1 Modul 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
3	xmlns:android="http://schemas.android.com/apk/res/an
4	droid"
5	xmlns:tools="http://schemas.android.com/tools"

6	android:id="@+id/mainLayout"
7	android:layout_width="match_parent"
8	android:layout_height="match_parent"
9	android:orientation="vertical"
10	android:gravity="center"
11	android:background="#FFFFFF"
12	android:padding="16dp"
13	tools:context=".MainActivity">
14	
15	<LinearLayout
16	android:layout_width="wrap_content"
17	android:layout_height="wrap_content"
18	android:orientation="horizontal"
19	android:gravity="center">
20	
21	<ImageView
22	android:id="@+id/dice_image1"
23	android:layout_width="150dp"
24	android:layout_height="150dp"
25	android:src="@drawable/dice_0"
26	android:layout_margin="8dp"/>
27	
28	<ImageView
29	android:id="@+id/dice_image2"
30	android:layout_width="150dp"
31	android:layout_height="150dp"
32	android:src="@drawable/dice_0"
33	android:layout_margin="8dp"/>
34	</LinearLayout>
35	

36	<Button
37	android:id="@+id/rollbuton"
38	android:layout_width="wrap_content"
39	android:layout_height="wrap_content"
40	android:text="ROLL DICE"
41	android:textColor="#FFFFFF"
42	android:backgroundTint="#00BCD4"
43	android:layout_marginTop="24dp"/>
44	</LinearLayout>

B. Output Program



Gambar 1. Screenshot Hasil Jawaban Soal 1 Modul 1

C. Pembahasan

1. MainActivity.kt:

Variabel `binding` digunakan untuk menginisialisasi `ViewBinding` dari layout `activity_main.xml`, sehingga seluruh elemen UI dapat diakses langsung tanpa menggunakan `findViewById`. Objek `viewModel` merupakan instance dari kelas

`DiceViewModel`, yang digunakan untuk menyimpan nilai dua dadu (`dice1` dan `dice2`) agar data tidak hilang saat terjadi perubahan konfigurasi, seperti rotasi layar. Pada fungsi `onCreate`, layout dihubungkan dengan aktivitas menggunakan `setContentView(binding.root)`, dan fungsi `updateDiceImages()` dipanggil untuk menampilkan nilai dadu terakhir yang disimpan di `ViewModel`. Saat tombol `ROLL DICE` ditekan, dua nilai acak dihasilkan dari objek `Dice`, lalu disimpan ke dalam `viewModel.dice1` dan `viewModel.dice2`. Fungsi `updateDiceImages()` digunakan untuk menampilkan gambar dadu berdasarkan hasil roll, serta menampilkan pesan melalui `Toast`. Jika kedua nilai dadu sama dan bukan nol, akan ditampilkan pesan “Selamat anda dapat dadu double!”, sedangkan jika berbeda, pesan yang muncul adalah “Anda belum beruntung!”. Fungsi `getDiceDrawable()` bertugas mengembalikan resource gambar dadu (`dice_1` hingga `dice_6`) sesuai angka hasil lemparan. Kelas `Dice` sendiri digunakan untuk menghasilkan angka acak dari 1 sampai jumlah sisi dadu, dengan default 6 sisi.

2. DaduView.kt

Kelas `DiceViewModel` merupakan turunan dari kelas `ViewModel`, yang digunakan untuk menyimpan data agar tetap bertahan saat terjadi perubahan konfigurasi, seperti rotasi layar. Di dalamnya terdapat dua variabel, yaitu `dice1` dan `dice2`, yang masing-masing menyimpan nilai lemparan dadu pertama dan kedua. Dengan menyimpan nilai ini di dalam `ViewModel`, aplikasi tetap dapat menampilkan hasil dadu terakhir tanpa ter-reset meskipun activity dibuat ulang.

3. activity_main.xml

Layout menggunakan elemen `LinearLayout` sebagai container utama dengan orientasi vertikal dan posisi elemen ditengah (`gravity="center"`), serta latar belakang putih dan padding sebesar 16dp. Di dalamnya terdapat `LinearLayout`

horizontal yang berfungsi untuk menampilkan dua gambar dadu secara sejajar. Setiap dadu ditampilkan menggunakan elemen `ImageView` dengan ukuran 150dp x 150dp dan jarak antar gambar diatur melalui atribut `layout_margin`. Gambar awal dadu diatur menggunakan `@drawable/dice_0`, yang merupakan gambar dadu kosong sebagai tampilan default. Di bawah gambar dadu terdapat elemen `Button` dengan ID `rollbutton`, berfungsi untuk memicu aksi "ROLL DICE". Tombol ini memiliki teks berwarna putih dan latar belakang berwarna biru kehijauan (`#00BCD4`), serta jarak atas sebesar 24dp agar tampilan lebih rapi.

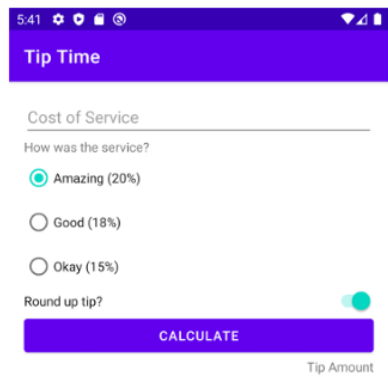
MODUL 2 : Android Layout

SOAL 1

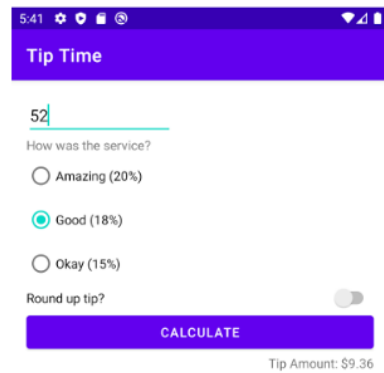
Soal Praktikum:

Buatlah sebuah aplikasi kalkulator tip yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

1. Input Biaya Layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
2. Pilihan Persentase Tip: Pengguna dapat memilih persentase tip yang diinginkan dari opsi yang disediakan, yaitu 15%, 18%, dan 20%.
3. Pengaturan Pembulatan Tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
4. Tampilan Hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 1 Tampilan Awal Aplikasi



Gambar 2 Tampilan Aplikasi Setelah Dijalankan

A. Source Code

1. MainActivity.kt

Tabel 4. Source Code Soal 1 Modul 2

1	package com.example.tipapp
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.activity.enableEdgeToEdge
7	import androidx.compose.foundation.background
8	import androidx.compose.foundation.layout.*
9	import androidx.compose.material3.*
10	import androidx.compose.runtime.Composable
11	import androidx.compose.ui.Alignment
12	import androidx.compose.ui.Modifier
13	import androidx.compose.ui.graphics.Color
14	import androidx.compose.ui.unit.dp
15	import androidx.compose.ui.unit.sp
16	import com.example.tipapp.ui.theme.TipAppTheme
17	import com.example.tipapp.ui.theme.TipScreen
18	
19	class MainActivity : ComponentActivity() {
20	override fun onCreate(savedInstanceState:
21	Bundle?) {
22	super.onCreate(savedInstanceState)
23	enableEdgeToEdge()
24	setContent {
25	TipAppTheme {

```

26         Surface(modifier =
27 Modifier.fillMaxSize()) {
28             TipTimeApp()
29         }
30     }
31 }
32 }
33 }
34
35 @Composable
36 fun TipTimeApp() {
37     Column(modifier = Modifier.fillMaxSize()) {
38         Box(
39             modifier = Modifier
40                 .fillMaxWidth()
41                 .height(56.dp)
42                 .background(Color(0xFF6200EE)),
43             contentAlignment = Alignment.Center
44         ) {
45             Text(
46                 text = "Tip Time",
47                 fontSize = 20.sp,
48                 color = Color.White
49             )
50         }
51         TipScreen(modifier =
52 Modifier.padding(16.dp))}}

```

2. TipScreen.kt

Tabel 5. Source Code Soal 1 Modul 2

1	package com.example.tipapp.ui.theme
2	
3	import android.widget.Toast
4	import androidx.compose.foundation.layout.*
5	import
6	androidx.compose.foundation.selection.selectable
7	import androidx.compose.material3.*
8	import androidx.compose.runtime.*
9	import
10	androidx.compose.runtime.saveable.rememberSaveable
11	import androidx.compose.ui.Alignment
12	import androidx.compose.ui.Modifier
13	import androidx.compose.ui.platform.LocalContext
14	import androidx.compose.ui.unit.dp
15	import
16	androidx.lifecycle.viewmodel.compose.viewModel
17	
18	@Composable
19	fun TipScreen(20 modifier: Modifier = Modifier, 21 viewModel: TipViewModel = viewModel() 22) { 23 val context = LocalContext.current 24 25 val amountInput by 26 viewModel.amountInput.collectAsState() 27 val tipPercent by 28 viewModel.tipPercent.collectAsState()

```

29     val roundUp by
30     viewModel.roundUp.collectAsState()
31
32     var showResult by rememberSaveable {
33     mutableStateOf(false) }
34     val tip = viewModel.calculateTip()
35
36     Column(
37         modifier = modifier
38             .fillMaxSize(),
39         verticalArrangement =
40     Arrangement.spacedBy(16.dp)
41     ) {
42         OutlinedTextField(
43             value = amountInput,
44             onChange = {
45     viewModel.onAmountChange(it) },
46             label = { Text("Cost of Service") },
47             modifier = Modifier.fillMaxWidth(),
48             singleLine = true
49         )
50
51         Text("How was the service?")
52
53         val options = listOf(
54             0.20 to "Amazing (20%)",
55             0.18 to "Good (18%)",
56             0.15 to "Okay (15%)"
57         )
58

```

```

59         options.forEach { (value, label) ->
60             Row(
61                 Modifier
62                     .fillMaxWidth()
63                     .selectable(
64                         selected = tipPercent ==
65 value,
66                         onClick = {
67 viewModel.onTipChange(value) }
68                     )
69                     .padding(vertical = 4.dp),
70                 verticalAlignment =
71 Alignment.CenterVertically
72             ) {
73                 RadioButton(
74                     selected = tipPercent == value,
75                     onClick = {
76 viewModel.onTipChange(value) }
77                 )
78                 Text(label)
79             }
80         }
81
82         Row(verticalAlignment =
83 Alignment.CenterVertically) {
84             Text("Round up tip?")
85             Spacer(modifier = Modifier.width(8.dp))
86             Switch(
87                 checked = roundUp,
88                 onCheckedChange = {

```

```

89     viewModel.onRoundUpChange(it) }
90         )
91     }
92
93     Button(
94         onClick = {
95             showResult =
96             amountInput.isNotBlank() &&
97             amountInput.toDoubleOrNull() != null
98             if (!showResult) {
99                 Toast.makeText(context, "Please
100 enter a valid amount", Toast.LENGTH_SHORT).show()
101             }
102         },
103         modifier = Modifier.fillMaxWidth(),
104         colors =
105         ButtonDefaults.buttonColors(containerColor =
106         MaterialTheme.colorScheme.primary)
107     ) {
108         Text("CALCULATE", color =
109         MaterialTheme.colorScheme.onPrimary)
110     }
111
112     if (showResult) {
113         Text(
114             text = "Tip Amount:
115             $%.2f".format(tip),
116             style =
117             MaterialTheme.typography.headlineSmall
118         )

```


119	}
120	}
121	}
122	
123	
124	

3. TipViewModel.kt

Tabel 6. Source Code Soal 1 Modul 2

1	package com.example.tipapp.ui.theme
2	
3	import androidx.lifecycle.ViewModel
4	import kotlinx.coroutines.flow.MutableStateFlow
5	import kotlinx.coroutines.flow.StateFlow
6	import kotlin.math.ceil
7	
8	class TipViewModel : ViewModel() {
9	private val _amountInput = MutableStateFlow("")
10	val amountInput: StateFlow<String> =
11	_amountInput
12	
13	private val _tipPercent = MutableStateFlow(0.20)
14	val tipPercent: StateFlow<Double> = _tipPercent
15	
16	private val _roundUp = MutableStateFlow(false)
17	val roundUp: StateFlow<Boolean> = _roundUp
18	
19	fun onAmountChange(newAmount: String) {
20	_amountInput.value = newAmount
21	}

```
22
23     fun onTipChange(percent: Double) {
24         _tipPercent.value = percent
25     }
26
27     fun onRoundUpChange(value: Boolean) {
28         _roundUp.value = value
29     }
30
31     fun calculateTip(): Double {
32         val amount =
33 _amountInput.value.toDoubleOrNull() ?: return 0.0
34         var result = amount * _tipPercent.value
35         if (_roundUp.value) result = ceil(result)
36         return result
37     }
38 }
39
40
```

B. Output Program

The image displays two side-by-side screenshots of a mobile application titled "Tip Time". Both screenshots show a purple header bar with the app name. The left screenshot shows the "Cost of Service" input field empty, "Amazing (20%)" selected for service quality, "Round up tip?" toggle on, and a "CALCULATE" button. The right screenshot shows "Cost of Service" as 52, "Good (18%)" selected, "Round up tip?" toggle off, and the result "Tip Amount: \$9,36" displayed below the "CALCULATE" button.

Gambar 2. Screenshot Hasil Jawaban Soal 1 Modul 2

C. Pembahasan

1. MainActivity.kt:

Kelas MainActivity adalah tempat pertama aplikasi dijalankan. Di dalam metode `onCreate()`, fungsi `enableEdgeToEdge()` digunakan agar tampilan aplikasi bisa tampil penuh hingga batas layar. Selanjutnya, seluruh antarmuka dibuat dengan Jetpack Compose melalui fungsi `setContent`, lalu dibungkus dengan `TipAppTheme` agar tampilan aplikasi mengikuti tema Material Design. Di dalamnya, fungsi `TipTimeApp()` digunakan untuk menampilkan dua bagian utama, yaitu AppBar berwarna ungu dengan judul "Tip Time", dan konten utama yang dipanggil melalui `TipScreen`, yaitu layar kalkulator tip. Di sini saya tidak menggunakan layout

XML, jadi seluruh UI dirancang langsung menggunakan Kotlin Compose, sehingga lebih modern dan efisien.

2. TipScreen.kt

Fungsi `TipScreen()` merupakan bagian utama UI yang menampilkan form kalkulator tip. Fungsi ini menampilkan input biaya layanan menggunakan `OutlinedTextField`, tiga pilihan rating layanan (Amazing, Good, Okay) menggunakan `RadioButton`, switch untuk membulatkan tip, dan tombol “CALCULATE” untuk menghitung hasil. Semua data (input angka, pilihan persentase, dan switch) dikelola oleh `ViewModel` agar tetap bertahan meskipun terjadi rotasi layar. Jika tombol ditekan tanpa input yang valid, akan muncul pesan toast. Saat input valid, hasil perhitungan tip ditampilkan dalam format dollar secara otomatis. Pendekatan ini menjadikan UI responsif, reaktif, dan tidak mudah ter-reset karena menggunakan `StateFlow` dan `rememberSaveable`.

3. TipViewModel.kt

`TipViewModel` adalah kelas yang mengatur dan menyimpan data aplikasi seperti input biaya, persentase tip, dan status pembulatan. Semua data disimpan dalam bentuk `StateFlow`, sehingga bisa diamati secara real-time di UI. Fungsi `onAmountChange()`, `onTipChange()`, dan `onRoundUpChange()` digunakan untuk memperbarui nilai-nilai tersebut saat pengguna berinteraksi. Fungsi `calculateTip()` bertugas menghitung nilai tip berdasarkan input pengguna dan akan membulatkan hasilnya jika opsi pembulatan diaktifkan. Dengan penggunaan `ViewModel` ini, data tetap aman meskipun pengguna merotasi layar atau sistem mereset ulang komponen UI.

MODUL 3 : Build a Scrollable List

SOAL 1

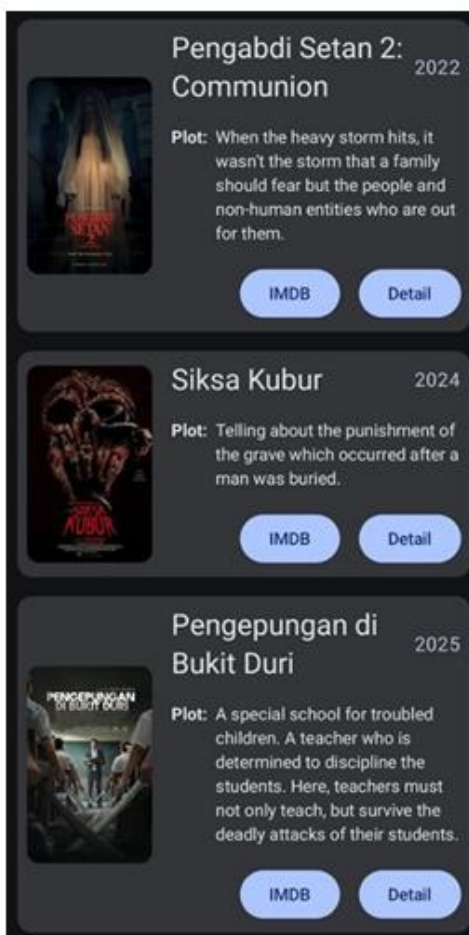
Soal Praktikum:

1. Buatlah sebuah aplikasi Android menggunakan XML atau Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:

1. List menggunakan fungsi RecyclerView (XML) atau LazyColumn (Compose)
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
4. Terdapat 2 button dalam list, dengan fungsi berikut:
 - a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
 - b. Button kedua menggunakan Navigation component/intent untuk membuka laman detail item
5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
8. Aplikasi berbasis XML harus menggunakan ViewBinding

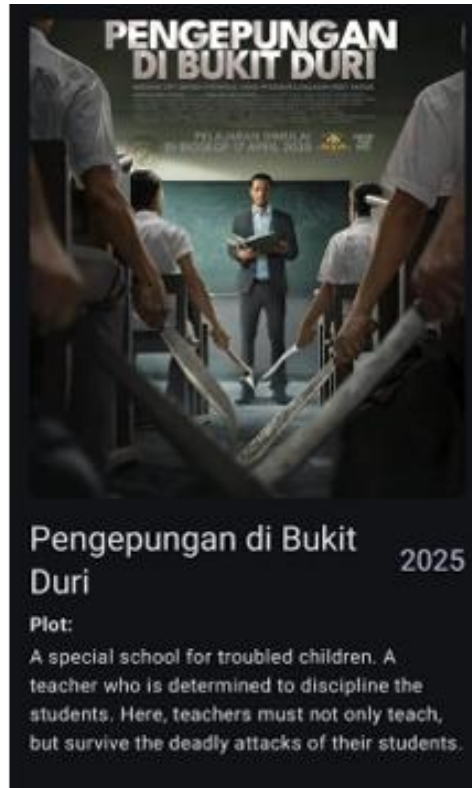
2. Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

UI item list harus berisi 1 gambar, 2 button (intent eksplisit dan navigasi), dan 2 baris teks dan setiap baris memiliki 2 teks yang berbeda. Diusahakan agar desain UI item list menyerupai UI berikut:



Gambar 1. Contoh UI List

Desain UI laman detail bebas, tetapi diusahakan untuk mengikuti kaidah desain Material Design dan data item ditampilkan penuh di laman detail seperti contoh berikut:



Gambar 2. Contoh UI Detail

A. Source Code

1. MainActivity.kt

Tabel 7. Source Code Soal 1 Modul 3

1	<code>package com.example.birthflowerzapp</code>
2	
3	<code>import android.os.Bundle</code>
4	<code>import androidx.activity.ComponentActivity</code>

5	import androidx.activity.compose.setContent
6	import androidx.compose.foundation.Image
7	import androidx.compose.foundation.layout.*
8	import
9	androidx.compose.foundation.shape.RoundedCornerShap
10	e
11	import androidx.compose.material3.*
12	import androidx.compose.runtime.Composable
13	import androidx.compose.ui.Modifier
14	import androidx.compose.ui.res.painterResource
15	import androidx.compose.ui.unit.dp
16	import
17	androidx.navigation.compose.rememberNavController
18	import androidx.navigation.compose.NavHost
19	import androidx.navigation.compose.composable
20	import androidx.navigation.NavController
21	import
22	com.example.birthflowerzapp.ui.theme.BirthFlowerzAp
23	pTheme
24	import androidx.compose.foundation.lazy.LazyColumn
25	import androidx.compose.foundation.lazy.items
26	import androidx.compose.ui.text.font.FontWeight
27	import androidx.compose.ui.unit.sp
28	import android.content.Intent
29	import android.net.Uri
30	import android.content.Context
31	import androidx.compose.ui.platform.LocalContext
32	import androidx.compose.ui.Alignment
33	
34	class MainActivity : ComponentActivity() {

35	override fun onCreate(savedInstanceState:
36	Bundle?) {
37	super.onCreate(savedInstanceState)
38	setContent {
39	BirthFlowerzAppTheme {
40	val navController =
41	rememberNavController()
42	
43	NavHost(navController =
44	navController, startDestination = "main_screen") {
45	composable("main_screen") {
46	MainScreen(navController =
47	navController)
48	}
49	
50	composable("flower_detail/{flowerId}") {
51	backStackEntry ->
52	val flowerId =
53	backStackEntry.arguments?.getString("flowerId")?.to
54	Int() ?: 0
55	FlowerDetailScreen(flower =
56	flowersList[flowerId])
57	}
58	}
59	}
60	}
61	}
62	}
63	
64	@Composable

65	fun mainScreen(navController: NavController) {
66	
67	val flowersList = listOf(
68	Flowers("January - Carnation",
69	"Symbolizes admiration and love.
70	Represents deep affection and distinction.",
71	
72	"https://en.wikipedia.org/wiki/Dianthus_caryophyllu
73	s",
74	R.drawable.carnation),
75	
76	Flowers("February - Iris",
77	"Represents wisdom and hope. Known for
78	elegance and royal symbolism.",
79	
80	"https://en.wikipedia.org/wiki/Iris_(plant)" ,
81	R.drawable.iris),
82	
83	Flowers("March - Daffodil",
84	"Symbol of new beginnings and hope.
85	Blooms mark the start of spring.",
86	
87	"https://en.wikipedia.org/wiki/Narcissus_(plant)" ,
88	R.drawable.daffodil),
89	
90	Flowers("April - Sweet Pea",
91	"Conveys bliss and goodbyes. Fragrant
92	flower representing gentle charm.",
93	
94	"https://en.wikipedia.org/wiki/Lathyrus_odoratus" ,

95	R.drawable.sweetpea),
96	
97	Flowers("May - Lily of the Valley",
98	"Represents purity and joy. White
99	blooms symbolize fresh starts.",
100	
101	"https://en.wikipedia.org/wiki/Lily_of_the_valley",
102	R.drawable.lilyofthevalley),
103	
104	Flowers("June - Rose",
105	"Classic symbol of love and beauty.
106	Each color has its own meaning.",
107	"https://en.wikipedia.org/wiki/Rose",
108	R.drawable.rose),
109	
110	Flowers("July - Larkspur",
111	"Stands for lightness and joy. Bright
112	spikes represent strong bonds.",
113	
114	"https://en.wikipedia.org/wiki/Delphinium",
115	R.drawable.larkspur),
116	
117	Flowers("August - Poppy",
118	"Symbolizes peace and remembrance. Red
119	petals evoke reflection.",
120	"https://en.wikipedia.org/wiki/Poppy",
121	R.drawable.poppy),
122	
123	Flowers("September - Aster",
124	"Represents love and faith. Star-shaped

```

125 petals show lasting beauty.",
126         "https://en.wikipedia.org/wiki/Aster",
127         R.drawable.aster),
128
129     Flowers("October - Marigold",
130         "Stands for warmth and creativity.
131 Golden tones bring optimism.",
132
133         "https://en.wikipedia.org/wiki/Tagetes",
134         R.drawable.marigold),
135
136     Flowers("November - Peony",
137         "Symbolizes romance and honor. Lush
138 blooms express elegance.",
139         "https://en.wikipedia.org/wiki/Peony",
140         R.drawable.peony),
141
142     Flowers("December - Narcissus",
143         "Represents self-love and rebirth. A
144 flower of renewal and growth.",
145
146         "https://en.wikipedia.org/wiki/Narcissus_(plant)",
147         R.drawable.narcissus)
148     )
149
150     LazyColumn(modifier = Modifier.padding(16.dp))
151 {
152     items(flowersList) { flower ->
153         FlowerItem(flower, navController)
154     }

```

155	}
156	}
157	
158	@Composable
159	fun FlowerItem(flower: Flowers, navController:
160	NavController) {
161	val context = LocalContext.current
162	
163	Card(
164	shape = RoundedCornerShape(8.dp),
165	modifier =
166	Modifier.fillMaxWidth().padding(8.dp)
167	) {
168	Row(modifier =
169	Modifier.fillMaxWidth().padding(16.dp)) {
170	Image(
171	painter = painterResource(id =
172	flower.image),
173	contentDescription = flower.name,
174	modifier = Modifier.size(120.dp)
175	)
176	Spacer(modifier =
177	Modifier.width(16.dp))
178	Column(
179	verticalArrangement =
180	Arrangement.Center,
181	modifier = Modifier.weight(1f)
182	) {
183	
184	Text(

```

185         text = flower.name,
186         style =
187     MaterialTheme.typography.headlineLarge.copy(fontWei
188     ght = FontWeight.Bold, fontSize = 22.sp)
189     )
190     Text(text = flower.symbol)
191     Row(
192         horizontalArrangement =
193     Arrangement.spacedBy(8.dp), // Add spacing between
194     buttons
195         modifier =
196     Modifier.fillMaxWidth(),
197         verticalAlignment =
198     Alignment.CenterVertically // Ensure both buttons
199     align vertically
200     ) {
201         Button(
202             onClick = {
203     openLink(flower.wikiLink, context) },
204             modifier =
205     Modifier.weight(1f) // Make buttons take equal
206     width
207         ) {
208             Text(text = "Learn More")
209         }
210         Button(
211             onClick = {
212
213     navController.navigate("flower_detail/${flowersList
214     .indexOf(flower)}

```

215	)
216	},
217	modifier =
218	Modifier.weight(1f) // Make buttons take equal
219	width
220	) {
221	Text(text = "Detail",
222	fontWeight = FontWeight.Bold)
223	}
224	}
225	}
226	}
227	}
228	}
229	
230	
231	fun openLink(url: String, context: Context) {
232	val intent = Intent(Intent.ACTION_VIEW,
233	Uri.parse(url))
234	context.startActivity(intent)
235	}
236	
237	
238	

2. Flowers.kt

Tabel 8. Source Code Soal 1 Modul 3

1	package com.example.birthflowerzapp
2	

3	import androidx.annotation.DrawableRes
4	
5	data class Flowers(
6	val name: String,
7	val symbol: String,
8	val wikiLink: String,
9	@DrawableRes val image: Int
10	)
11	
12	val <i>flowersList</i> = listOf(
13	Flowers("January - Carnation",
14	"Symbolizes admiration and love. Represents
15	deep affection and distinction.",
16	
17	"https://en.wikipedia.org/wiki/Dianthus_caryophyllus
18	",
19	R.drawable.carnation),
20	
21	Flowers("February - Iris",
22	"Represents wisdom and hope. Known for
23	elegance and royal symbolism.",
24	
25	"https://en.wikipedia.org/wiki/Iris_(plant)",
26	R.drawable.iris),
27	
28	Flowers("March - Daffodil",
29	"Symbol of new beginnings and hope. Blooms
30	mark the start of spring.",
31	
32	"https://en.wikipedia.org/wiki/Narcissus_(plant)",

33	<code>R.drawable.daffodil),</code>
34	
35	<code>Flowers("April - Sweet Pea",</code>
36	<code>"Conveys bliss and goodbyes. Fragrant flower</code>
37	<code>representing gentle charm.",</code>
38	
39	<code>"https://en.wikipedia.org/wiki/Lathyrus_odoratus",</code>
40	<code>R.drawable.sweetpea),</code>
41	
42	<code>Flowers("May - Lily of the Valley",</code>
43	<code>"Represents purity and joy. White blooms</code>
44	<code>symbolize fresh starts.",</code>
45	
46	<code>"https://en.wikipedia.org/wiki/Lily_of_the_valley",</code>
47	<code>R.drawable.lilyofthevalley),</code>
48	
49	<code>Flowers("June - Rose",</code>
50	<code>"Classic symbol of love and beauty. Each</code>
51	<code>color has its own meaning.",</code>
52	<code>"https://en.wikipedia.org/wiki/Rose",</code>
53	<code>R.drawable.rose),</code>
54	
55	<code>Flowers("July - Larkspur",</code>
56	<code>"Stands for lightness and joy. Bright spikes</code>
57	<code>represent strong bonds.",</code>
58	<code>"https://en.wikipedia.org/wiki/Delphinium",</code>
59	<code>R.drawable.larkspur),</code>
60	
61	<code>Flowers("August - Poppy",</code>
62	<code>"Symbolizes peace and remembrance. Red</code>

63	petals evoke reflection.",
64	"https://en.wikipedia.org/wiki/Poppy" ,
65	R.drawable.poppy),
66	
67	Flowers("September - Aster",
68	"Represents love and faith. Star-shaped
69	petals show lasting beauty.",
70	"https://en.wikipedia.org/wiki/Aster" ,
71	R.drawable.aster),
72	
73	Flowers("October - Marigold",
74	"Stands for warmth and creativity. Golden
75	tones bring optimism.",
76	"https://en.wikipedia.org/wiki/Tagetes" ,
77	R.drawable.marigold),
78	
79	Flowers("November - Peony",
80	"Symbolizes romance and honor. Lush blooms
81	express elegance.",
82	"https://en.wikipedia.org/wiki/Peony" ,
83	R.drawable.peony),
84	
85	Flowers("December - Narcissus",
86	"Represents self-love and rebirth. A flower
87	of renewal and growth.",
88	
89	"https://en.wikipedia.org/wiki/Narcissus_(plant)" ,
90	R.drawable.narcissus)
91	)

3. FlowerDetailActivity.kt

Tabel 9. Source Code Soal 1 Modul 3

1	package com.example.birthflowerzapp
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.compose.foundation.Image
7	import androidx.compose.foundation.layout.*
8	import androidx.compose.foundation.lazy.LazyColumn
9	import androidx.compose.foundation.lazy.items
10	import androidx.compose.material3.*
11	import androidx.compose.runtime.Composable
12	import androidx.compose.ui.Modifier
13	import androidx.compose.ui.res.painterResource
14	import androidx.compose.ui.unit.dp
15	import androidx.compose.ui.text.font.FontWeight
16	import androidx.compose.ui.unit.sp
17	import androidx.compose.ui.Alignment
18	
19	class FlowerDetailActivity : ComponentActivity() {
20	override fun onCreate(savedInstanceState:
21	Bundle?) {
22	super.onCreate(savedInstanceState)
23	setContent {
24	val flowerId =
25	intent.getIntExtra("flowerId", 0)
26	val flower = flowersList[flowerId]
27	FlowerDetailScreen(flower = flower)
28	}

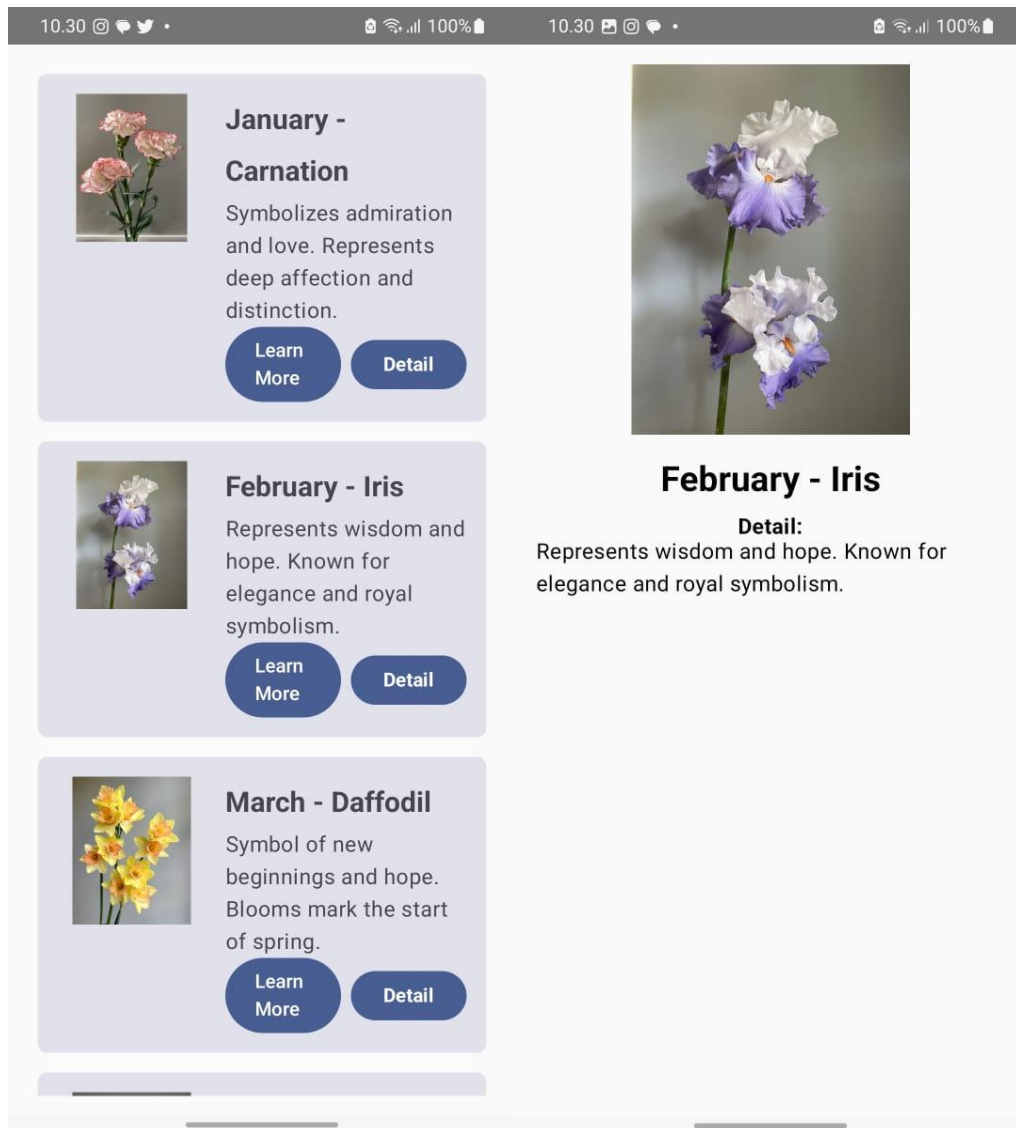
```

29     }
30 }
31
32 @Composable
33 fun FlowerDetailScreen(flower: Flowers) {
34     LazyColumn(
35         modifier = Modifier
36             .padding(16.dp)
37             .fillMaxSize(),
38         verticalArrangement = Arrangement.Top,
39         horizontalAlignment =
40 Alignment.CenterHorizontally
41     ) {
42         items(listOf(flower)) { currentFlower ->
43             Image(
44                 painter = painterResource(id =
45 currentFlower.image),
46                 contentDescription =
47 currentFlower.name,
48                 modifier = Modifier.size(300.dp)
49             )
50             Spacer(modifier =
51 Modifier.height(16.dp))
52
53             Text(
54                 text = currentFlower.name,
55                 style =
56 MaterialTheme.typography.headlineLarge.copy(fontWeig
57 ht = FontWeight.Bold, fontSize = 28.sp)
58             )

```

59	Spacer(modifier = Modifier.height(8.dp))
60	
61	Text(text = "Detail:", style =
62	MaterialTheme.typography.bodyLarge.copy(fontWeight =
63	FontWeight.Bold))
64	
65	Text(text = currentFlower.symbol, style
66	= MaterialTheme.typography.bodyLarge)
67	}
68	}
69	}
70	
71	

B. Output Program



Gambar 3 . Screenshot Hasil Jawaban Soal 1 Modul 3

C. Pembahasan

1. MainActivity.kt:

- MainActivity:

- **MainActivity** adalah layar utama aplikasi yang menampilkan daftar bunga berdasarkan bulan. Kode ini menggunakan Jetpack Compose untuk membangun UI.
- **NavController** digunakan untuk mengelola navigasi antar halaman. Pada halaman utama (`main_screen`), kita menampilkan daftar bunga dengan **LazyColumn**.
- **NavHost** mengelola navigasi antar halaman, di mana ada dua halaman: `main_screen` untuk menampilkan daftar bunga, dan `flower_detail` untuk menampilkan detail bunga.

- **MainScreen:**

- **LazyColumn** digunakan untuk menampilkan daftar bunga secara efisien. Di sini, setiap bunga ditampilkan dengan gambar, nama, deskripsi singkat, dan dua tombol.
- Setiap item pada daftar bunga adalah objek `FlowerItem` yang berisi informasi tentang bunga dan dua tombol:
 - **"Learn More"**: Mengarahkan ke halaman Wikipedia bunga menggunakan fungsi `openLink()`.
 - **"Detail"**: Menavigasi ke halaman detail bunga dengan mengirimkan ID bunga menggunakan `navController.navigate()`.

- **FlowerItem:**

- Menampilkan gambar bunga, nama bunga, dan deskripsi singkat.
- Dua tombol pada setiap item memungkinkan pengguna untuk:
 - Membuka link Wikipedia bunga.
 - Menavigasi ke halaman detail bunga.

- **Navigasi:**

- Halaman detail (`flower_detail`) akan menampilkan informasi lengkap bunga berdasarkan ID bunga yang dipilih dari daftar.
- ID bunga dipassing melalui URL menggunakan `navController.navigate("flower_detail/{flowerId}")`, sehingga halaman detail menerima data bunga yang sesuai berdasarkan ID.

- **Fungsi `openLink`:**

- Fungsi ini membuka link URL (dalam hal ini, Wikipedia) menggunakan **`Intent`** untuk memulai browser dan menampilkan halaman web terkait bunga.

Secara keseluruhan, aplikasi ini menampilkan daftar bunga berdasarkan bulan dengan navigasi antar halaman menggunakan Jetpack Compose dan **`NavController`** untuk routing antar halaman. Pengguna dapat memilih bunga dari daftar dan melihat detail lebih lanjut atau membuka halaman Wikipedia terkait bunga tersebut.

2. `Flowers.kt`

Kode ini mendefinisikan sebuah **`data class`** dan **`list`** yang menyimpan data bunga berdasarkan bulan. Berikut penjelasan singkatnya:

- **Data Class `Flowers`:**

- `Flowers` adalah kelas yang menyimpan informasi tentang bunga. Setiap objek `Flowers` memiliki:
 - `name`: Nama bunga (misalnya, "January - Carnation").
 - `symbol`: Deskripsi singkat tentang bunga tersebut.
 - `wikiLink`: Link ke Wikipedia untuk informasi lebih lanjut.

- `image`: ID gambar bunga yang disimpan dalam sumber daya drawable.

- **List flowersList:**

- `flowersList` adalah sebuah **list** yang berisi objek `Flowers` untuk setiap bulan dengan data bunga masing-masing. Setiap bulan memiliki bunga dengan nama, deskripsi, link, dan gambar.
- Misalnya, untuk bulan Januari, bunga yang dipilih adalah `Carnation` (Anyelir), dengan deskripsi, link ke Wikipedia, dan gambar bunga.

Secara keseluruhan, kode ini digunakan untuk mengelola data bunga berdasarkan bulan, lengkap dengan gambar dan link informasi terkait.

3. FlowerDetailActivity.kt

- **FlowerDetailActivity:**

- **FlowerDetailActivity** adalah aktivitas yang menampilkan detail dari sebuah bunga. Pada saat halaman ini dibuka, **flowerId** diambil dari intent dan digunakan untuk mendapatkan objek `Flower` yang sesuai dari `flowersList`.
- Setelah mendapatkan bunga yang dipilih, data bunga tersebut diteruskan ke **FlowerDetailScreen** untuk ditampilkan.

- **FlowerDetailScreen:**

- **LazyColumn** digunakan untuk menampilkan informasi bunga secara vertikal dan dapat digulir (scrollable). Di dalamnya, hanya ada satu item bunga yang ditampilkan.

- **Image:** Gambar bunga ditampilkan menggunakan `painterResource` berdasarkan ID gambar yang terdapat dalam objek `Flower`.
- **Text:** Nama bunga dan deskripsi singkat (symbol) bunga ditampilkan dengan font besar dan bold untuk nama bunga dan gaya normal untuk deskripsi.
- **Spacer** digunakan untuk memberikan jarak antar elemen seperti gambar dan teks agar tampak lebih rapi.

Kesimpulan:

Kode ini membuat halaman detail bunga yang menampilkan gambar, nama, dan deskripsi singkat dari bunga yang dipilih. Data bunga diproses dari **FlowerDetailActivity** dan diteruskan ke **FlowerDetailScreen** untuk ditampilkan menggunakan `LazyColumn` agar bisa digulir dengan nyaman.

2. Mengapa `RecyclerView` masih digunakan, padahal `RecyclerView` memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan `LazyColumn` dengan kode yang lebih singkat?

JAWAB: `RecyclerView` masih digunakan karena memiliki fleksibilitas tinggi, dukungan luas pada berbagai versi Android, dan ekosistem yang matang, serta cocok untuk proyek berbasis View System. `LazyColumn` hanya tersedia di Jetpack Compose, yang belum sepenuhnya diadopsi di semua proyek.

Referensi:

<https://developer.android.com/develop/ui/compose/lists>

<https://developer.android.com/develop/ui/views/layout/recyclerview>

MODUL 4 : ViewModel and Debugging

SOAL 1

Soal Praktikum:

1. Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.

b. Gunakan ViewModelFactory dalam pembuatan ViewModel

c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment

d. gunakan logging untuk event berikut:

a. Log saat data item masuk ke dalam list

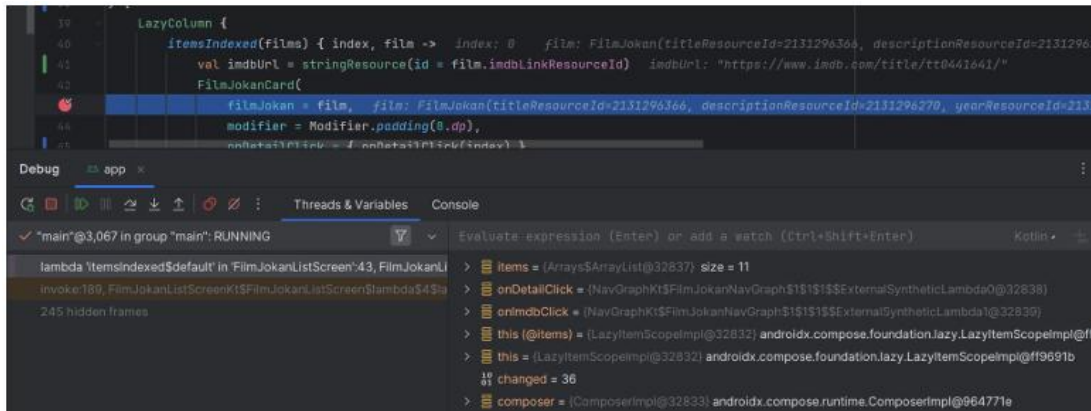
b. Log saat tombol Detail dan tombol Explicit Intent ditekan

c. Log data dari list yang dipilih ketika berpindah ke halaman Detail

e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out

2. Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

Aplikasi harus dapat mempertahankan fitur-fitur yang sudah dibuat pada modul sebelumnya. Berikut adalah contoh debugging dalam Android Studio.



Contoh Penggunaan Debugger

A. Source Code

1. MainActivity.kt

Tabel 10. Source Code Soal 1 Modul 4

1	package com.example.birthflowerzapp
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.compose.foundation.Image
7	import androidx.compose.foundation.layout.*
8	import
9	androidx.compose.foundation.shape.RoundedCornerShap
10	e
11	import androidx.compose.material3.*
12	import androidx.compose.runtime.Composable
13	import androidx.compose.ui.Modifier
14	import androidx.compose.ui.res.painterResource
15	import androidx.compose.ui.unit.dp

16	import
17	androidx.navigation.compose.rememberNavController
18	import androidx.navigation.compose.NavHost
19	import androidx.navigation.compose.composable
20	import androidx.navigation.NavController
21	import
22	com.example.birthflowerzapp.ui.theme.BirthFlowerzApp
23	Theme
24	import androidx.compose.foundation.lazy.LazyColumn
25	import androidx.compose.foundation.lazy.items
26	import androidx.compose.ui.text.font.FontWeight
27	import androidx.compose.ui.unit.sp
28	import android.content.Intent
29	import android.net.Uri
30	import android.content.Context
31	import androidx.compose.ui.platform.LocalContext
32	import androidx.compose.ui.Alignment
33	import
34	androidx.lifecycle.viewmodel.compose.viewModel
35	import androidx.compose.runtime.collectAsState
36	import
37	com.example.birthflowerzapp.BirthFlowerViewModel
38	import
39	com.example.birthflowerzapp.BirthFlowerViewModelFac
40	tory
41	
42	class MainActivity : ComponentActivity() {
43	override fun onCreate(savedInstanceState:
44	Bundle?) {
45	super.onCreate(savedInstanceState)

```

46         setContent {
47             BirthFlowerzAppTheme {
48                 val navController =
49 rememberNavController()
50
51                 NavHost(navController =
52 navController, startDestination = "main_screen") {
53                     composable("main_screen") {
54                         MainScreen(navController =
55 navController)
56                     }
57
58 composable("flower_detail/{flowerId}") {
59 backStackEntry ->
60                     val flowerId =
61 backStackEntry.arguments?.getString("flowerId")?.to
62 Int() ?: 0
63                     FlowerDetailScreen(flower =
64 flowersList[flowerId])
65                 }
66             }
67         }
68     }
69 }
70 }
71
72 @Composable
73 fun MainScreen(navController: NavController) {
74     val viewModel: BirthFlowerViewModel =
75 viewModel(factory = BirthFlowerViewModelFactory())

```

```

76         val flowers =
77     viewModel.flowers.collectAsState()
78
79     LazyColumn(modifier = Modifier.padding(16.dp))
80 {
81     items(flowers.value.size) { index ->
82         val flower = flowers.value[index]
83         FlowerItem(
84             flower = flower,
85             index = index,
86             viewModel = viewModel,
87             navController = navController
88         )
89     }
90 }
91 }
92
93 @Composable
94 fun FlowerItem(
95     flower: Flowers,
96     index: Int,
97     viewModel: BirthFlowerViewModel,
98     navController: NavController
99 ) {
100     val context = LocalContext.current
101
102     Card(
103         shape = RoundedCornerShape(8.dp),
104         modifier = Modifier
105             .fillMaxWidth()

```

106	.padding(8.dp)
107	) {
108	Row(
109	modifier = Modifier
110	.fillMaxWidth()
111	.padding(16.dp),
112	verticalAlignment =
113	Alignment.CenterVertically
114	) {
115	Image(
116	painter = painterResource(id =
117	flower.image),
118	contentDescription = flower.name,
119	modifier = Modifier
120	.size(140.dp)
121	)
122	
123	Spacer(modifier =
124	Modifier.width(16.dp))
125	
126	Column(
127	modifier = Modifier.weight(1f)
128	) {
129	Text(
130	text = flower.name,
131	style =
132	MaterialTheme.typography.headlineMedium.copy(
133	fontWeight =
134	FontWeight.Bold
135	)

136	)
137	
138	Spacer(modifier =
139	Modifier.height(4.dp))
140	
141	Text(
142	text = "Detail:",
143	style =
144	MaterialTheme.typography.bodyMedium.copy(
145	fontWeight =
146	FontWeight.Bold
147	)
148	)
149	
150	Spacer(modifier =
151	Modifier.height(4.dp))
152	
153	Text(
154	text = flower.symbol,
155	style =
156	MaterialTheme.typography.bodyMedium
157	)
158	
159	Spacer(modifier =
160	Modifier.height(8.dp))
161	
162	Row(
163	horizontalArrangement =
164	Arrangement.spacedBy(8.dp)
165	) {

166	Button(
167	onClick = {
168	openLink(flower.wikiLink, context) },
169	modifier = Modifier
170	.width(75.dp)
171	) {
172	Text(
173	text = "Wiki",
174	fontSize = 12.sp,
175	maxLines = 1
176	)
177	}
178	
179	Button(
180	onClick = {
181	
182	viewModel.selectFlower(index) // Logging klik
183	detail
184	
185	navController.navigate("flower_detail/\$index")
186	},
187	modifier = Modifier
188	.width(90.dp)
189	) {
190	Text(
191	text = "Detail",
192	fontWeight =
193	FontWeight.Bold,
194	fontSize = 12.sp,
195	maxLines = 1

196	)
197	}
198	}
199	}
200	}
201	}
202	}
203	
204	fun openLink(url: String, context: Context) {
205	val intent = Intent(Intent.ACTION_VIEW,
206	Uri.parse(url))
207	context.startActivity(intent)
208	}

2. Flowers.kt

Tabel 11. Source Code Soal 1 Modul 4

1	package com.example.birthflowerzapp
2	
3	import androidx.annotation.DrawableRes
4	
5	data class Flowers(
6	val name: String,
7	val symbol: String,
8	val wikiLink: String,
9	@DrawableRes val image: Int
10	)
11	
12	val flowersList = listOf(
13	Flowers("January - Carnation",

14	"Symbolizes admiration and love. Represents
15	deep affection and distinction.",
16	
17	"https://en.wikipedia.org/wiki/Dianthus_caryophyllus
18	",
19	R.drawable.carnation),
20	
21	Flowers("February - Iris",
22	"Represents wisdom and hope. Known for
23	elegance and royal symbolism.",
24	
25	"https://en.wikipedia.org/wiki/Iris_(plant)",
26	R.drawable.iris),
27	
28	Flowers("March - Daffodil",
29	"Symbol of new beginnings and hope. Blooms
30	mark the start of spring.",
31	
32	"https://en.wikipedia.org/wiki/Narcissus_(plant)",
33	R.drawable.daffodil),
34	
35	Flowers("April - Sweet Pea",
36	"Conveys bliss and goodbyes. Fragrant flower
37	representing gentle charm.",
38	
39	"https://en.wikipedia.org/wiki/Lathyrus_odoratus",
40	R.drawable.sweetpea),
41	
42	Flowers("May - Lily of the Valley",
43	

44	"Represents purity and joy. White blooms
45	symbolize fresh starts.",
46	
47	"https://en.wikipedia.org/wiki/Lily_of_the_valley",
48	R.drawable.lilyofthevalley),
49	
50	Flowers("June - Rose",
51	"Classic symbol of love and beauty. Each
52	color has its own meaning.",
53	"https://en.wikipedia.org/wiki/Rose",
54	R.drawable.rose),
55	
56	Flowers("July - Larkspur",
57	"Stands for lightness and joy. Bright spikes
58	represent strong bonds.",
59	"https://en.wikipedia.org/wiki/Delphinium",
60	R.drawable.larkspur),
61	
62	Flowers("August - Poppy",
63	"Symbolizes peace and remembrance. Red
64	petals evoke reflection.",
65	"https://en.wikipedia.org/wiki/Poppy",
66	R.drawable.poppy),
67	
68	Flowers("September - Aster",
69	"Represents love and faith. Star-shaped
70	petals show lasting beauty.",
71	"https://en.wikipedia.org/wiki/Aster",
72	R.drawable.aster),
73	

74	Flowers("October - Marigold",
75	"Stands for warmth and creativity. Golden
76	tones bring optimism.",
77	"https://en.wikipedia.org/wiki/Tagetes",
78	R.drawable.marigold),
79	
80	Flowers("November - Peony",
81	"Symbolizes romance and honor. Lush blooms
82	express elegance.",
83	"https://en.wikipedia.org/wiki/Peony",
84	R.drawable.peony),
85	
86	Flowers("December - Narcissus",
87	"Represents self-love and rebirth. A flower
88	of renewal and growth.",
89	
90	"https://en.wikipedia.org/wiki/Narcissus_(plant)",
91	R.drawable.narcissus)
92	)
93	

3. FlowerDetailActivity.kt

Tabel 12. Source Code Soal 1 Modul 4

1	package com.example.birthflowerzapp
2	
3	import android.os.Bundle
4	import android.util.Log
5	import androidx.activity.ComponentActivity
6	import androidx.activity.compose.setContent

```

7 import androidx.compose.foundation.Image
8 import androidx.compose.foundation.layout.*
9 import androidx.compose.foundation.lazy.LazyColumn
10 import androidx.compose.foundation.lazy.items
11 import androidx.compose.material3.*
12 import androidx.compose.runtime.Composable
13 import androidx.compose.ui.Modifier
14 import androidx.compose.ui.res.painterResource
15 import androidx.compose.ui.unit.dp
16 import androidx.compose.ui.text.font.FontWeight
17 import androidx.compose.ui.unit.sp
18 import androidx.compose.ui.Alignment
19 import androidx.compose.ui.text.style.TextAlign
20
21
22 class FlowerDetailActivity : ComponentActivity() {
23     override fun onCreate(savedInstanceState:
24 Bundle?) {
25         super.onCreate(savedInstanceState)
26         setContent {
27             val flowerId =
28 intent.getIntExtra("flowerId", 0)
29             val flower = flowersList[flowerId]
30             FlowerDetailScreen(flower = flower)
31         }
32     }
33 }
34
35 @Composable
36 fun FlowerDetailScreen(flower: Flowers) {

```

37	Log.d("FlowerDetailScreen", "Menampilkan detail
38	bunga: \${flower.name}")
39	
40	LazyColumn(
41	modifier = Modifier
42	.padding(16.dp)
43	.fillMaxSize(),
44	verticalArrangement = Arrangement.Top,
45	horizontalAlignment =
46	Alignment.CenterHorizontally
47	) {
48	items(listOf(flower)) { currentFlower ->
49	Image(
50	painter = painterResource(id =
51	currentFlower.image),
52	contentDescription =
53	currentFlower.name,
54	modifier = Modifier.size(300.dp)
55	)
56	Spacer(modifier =
57	Modifier.height(16.dp))
58	
59	Text(
60	text = currentFlower.name,
61	style =
62	MaterialTheme.typography.headlineLarge.copy(
63	fontWeight = FontWeight.Bold,
64	fontSize = 28.sp
65	)
66	)

67	Spacer(modifier = Modifier.height(8.dp))
68	
69	Text(
70	text = "Detail:",
71	style =
72	MaterialTheme.typography.bodyLarge.copy(
73	fontWeight = FontWeight.Bold
74	),
75	modifier = Modifier.fillMaxWidth(),
76	textAlign = TextAlign.Start
77	)
78	
79	Spacer(modifier = Modifier.height(4.dp))
80	
81	Text(
82	text = flower.symbol,
83	style =
84	MaterialTheme.typography.bodyLarge
85	)
86	}
87	}
88	}
89	
90	
91	
92	
93	
94	

4. BirthFlowerViewModel.kt

Tabel 13. Source Code Soal 1 Modul 4

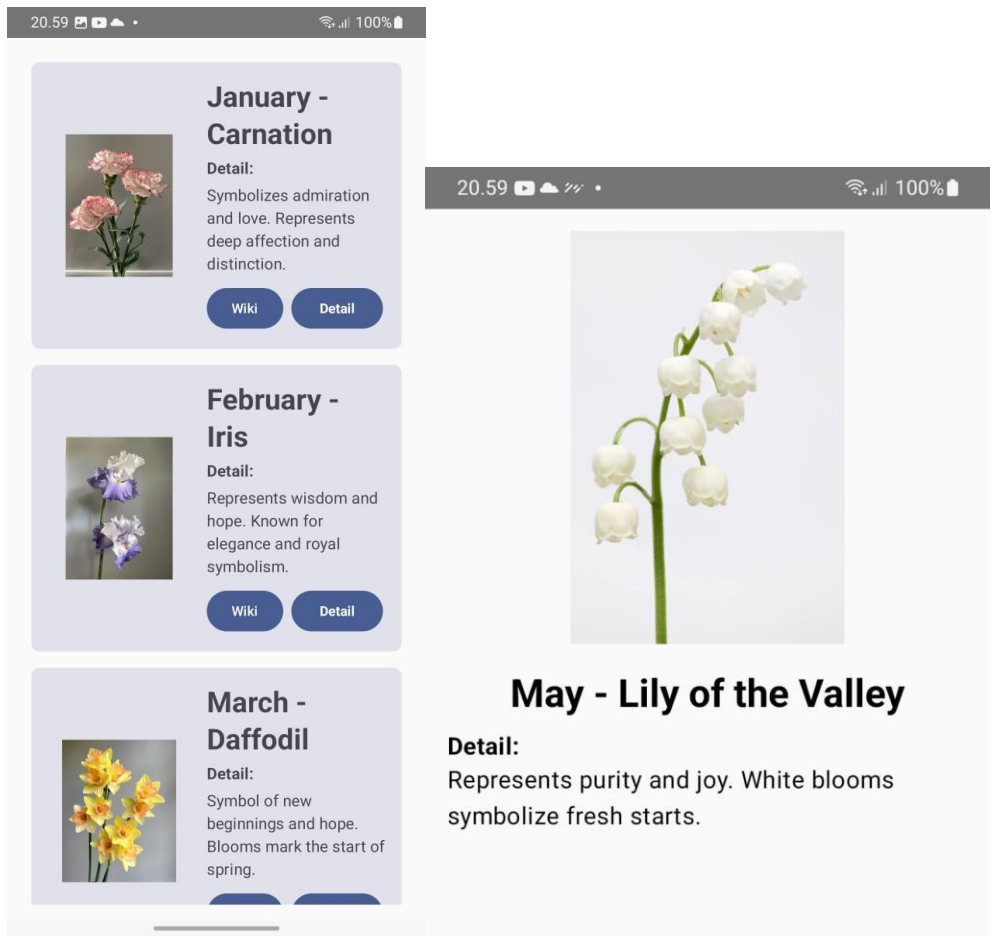
1	package com.example.birthflowerzapp
2	
3	import androidx.lifecycle.ViewModel
4	import kotlinx.coroutines.flow.MutableStateFlow
5	import kotlinx.coroutines.flow.StateFlow
6	import android.util.Log
7	
8	class BirthFlowerViewModel : ViewModel() {
9	
10	private val _flowers =
11	MutableStateFlow(flowersList)
12	val flowers: StateFlow<List<Flowers>> get() =
13	_flowers
14	
15	private val _selectedFlowerIndex =
16	MutableStateFlow<Int?>(null)
17	val selectedFlowerIndex: StateFlow<Int?> get() =
18	_selectedFlowerIndex
19	
20	fun selectFlower(index: Int) {
21	_selectedFlowerIndex.value = index
22	Log.d("BirthFlowerViewModel", "Tombol Detail
23	ditekan untuk bunga: \${flowersList[index].name}")
24	}
25	
26	init {
27	Log.d("BirthFlowerViewModel", "Data bunga
28	ditambahkan: \${flowersList.size} item")}}

5. BirthFlowerViewModelFactory.kt

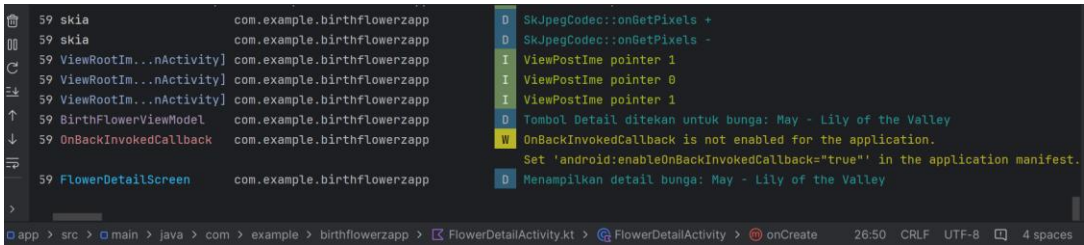
Tabel 14. Source Code Soal 1 Modul 4

1	package com.example.birthflowerzapp
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	
6	class BirthFlowerViewModelFactory :
7	ViewModelProvider.Factory {
8	override fun <T : ViewModel> create(modelClass:
9	Class<T>): T {
10	if
11	(modelClass.isAssignableFrom(BirthFlowerViewModel::c
12	lass.java)) {
13	return BirthFlowerViewModel() as T
14	}
15	throw IllegalArgumentException("Unknown
16	ViewModel class")
17	}
18	}
19	
20	

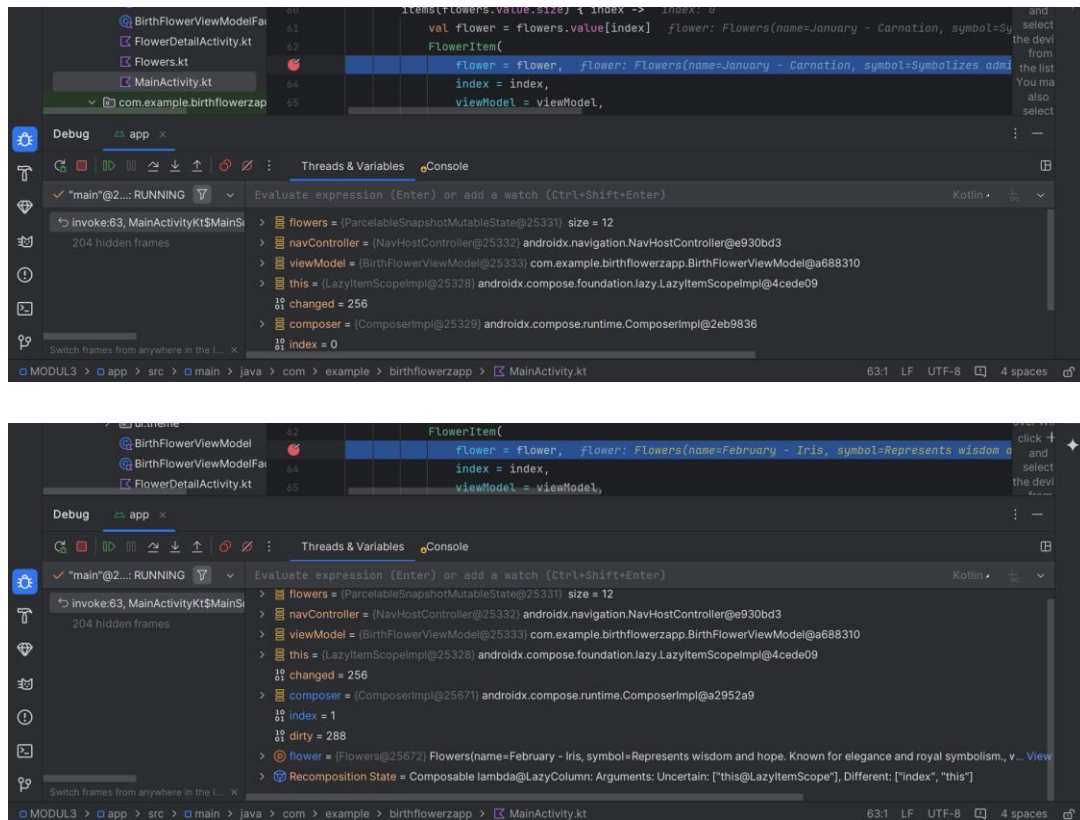
B. Output Program



Gambar 4. Screenshot Tampilan Aplikasi



Gambar 5. Screenshot Hasil Logging



Gambar 6. Screenshot Penggunaan Debugger

1. Debugger adalah alat bantu dalam Android Studio yang digunakan untuk:

- Menjalankan aplikasi baris per baris
- Melihat isi variabel secara langsung saat aplikasi berjalan
- Membantu menemukan bug atau kesalahan logika dalam kode

2. Cara Menggunakan Debugger:

- Tambahkan breakpoint (titik merah) di baris kode penting, misalnya saat fungsi dijalankan.
- Jalankan aplikasi dengan menekan tombol Debug.

- Saat eksekusi mencapai breakpoint, aplikasi akan berhenti sementara (pause).
- Gunakan fitur Step untuk menelusuri alur program.

3. Fitur Step:

- Step Into: Masuk ke dalam fungsi yang dipanggil untuk melihat isi detailnya.
- Step Over: Melewati fungsi tanpa masuk ke dalamnya, lanjut ke baris berikutnya.
- Step Out: Keluar dari fungsi sekarang dan kembali ke tempat fungsi itu dipanggil.

Pada gambar screenshoot debug saya, breakpoint aktif di MainActivity.kt saat FlowerItem() dijalankan:

- Variabel seperti index, flower, dan viewModel bisa dilihat nilainya langsung.
- Fitur Step Into, Step Over, dan Step Out bisa digunakan untuk melanjutkan eksekusi program sambil menelusuri logika kode.

C. Pembahasan

1. MainActivity.kt:

Ini adalah file utama yang menjalankan aplikasi dan menampilkan antarmuka pengguna. Di dalamnya terdapat onCreate() yang memanggil setContent untuk menampilkan tampilan berbasis **Jetpack Compose**. Aplikasi ini menggunakan **navigasi Compose** untuk berpindah antar halaman. Di dalam NavHost, terdapat dua route: main_screen dan flower_detail/{flowerId}. Halaman main_screen menampilkan daftar bunga, sedangkan halaman flower_detail menampilkan detail bunga berdasarkan ID yang dikirim. Fungsi mainScreen

mengambil data dari **ViewModel** dengan cara memanggil `viewModel(factory = BirthFlowerViewModelFactory())`. Data bunga yang tersimpan dalam **ViewModel** disimpan menggunakan **StateFlow**, lalu ditampilkan ke layar dengan `collectAsState()`. Data tidak lagi disimpan langsung di **Activity**, melainkan dikelola secara terpisah oleh **ViewModel** agar lebih aman dan tetap tersimpan saat layar diputar atau navigasi berpindah. Setiap item bunga ditampilkan dengan fungsi `FlowerItem`, yang berisi gambar, nama bunga, deskripsi singkat, dan dua tombol. Tombol "Wiki" membuka halaman Wikipedia menggunakan `Intent`, sedangkan tombol "Detail" memanggil fungsi `selectFlower(index)` dari **ViewModel** dan berpindah ke halaman detail. Di sinilah praktik **logging dan debugging** mulai diterapkan, karena saat tombol ini ditekan, log dicatat di Logcat, dan kita bisa memasang breakpoint di bagian ini saat debugging.

2. Flowers.kt

Kode ini berisi data class `Flowers` yang mendefinisikan struktur data bunga, yaitu `name`, `symbol`, `wikiLink`, dan `image`. Di bawahnya terdapat `flowersList`, yaitu daftar lengkap bunga dari bulan Januari hingga Desember, masing-masing dengan informasi nama, arti, link Wikipedia, dan gambar yang tersimpan di `drawable`. Data ini sebelumnya langsung digunakan di **MainActivity**, tetapi setelah penerapan **Modul 4**, list ini dipindahkan ke dalam **ViewModel** agar lebih aman dan sesuai prinsip **pemisahan logika dan tampilan (separation of concern)**.

3. FlowerDetailActivity.kt

File ini digunakan untuk menampilkan detail bunga ketika pengguna mengklik tombol "Detail". Di dalam fungsi `onCreate`, aplikasi mengambil `flowerId` dari `intent`, lalu menggunakan ID tersebut untuk mengambil objek bunga dari daftar. Bagian utama tampilannya ditulis dalam fungsi `FlowerDetailScreen`, yang menampilkan

gambar bunga, nama bunga, dan penjelasannya. Di awal fungsi ini juga terdapat perintah `Log.d(...)` untuk mencatat bahwa pengguna sedang melihat detail bunga tertentu. Log ini penting untuk keperluan **debugging**, karena kita bisa memantau interaksi pengguna dan menelusuri masalah jika ada tampilan atau data yang tidak sesuai.

4. BirthFlowerViewModel.kt

File ini berisi list bunga (`flowersList`) disimpan dalam sebuah **StateFlow** bernama `_flowers`, sehingga data bisa di-observe dan diperbarui jika perlu. Selain itu, ViewModel ini juga menyimpan `selectedFlowerIndex` — index bunga yang terakhir dipilih oleh pengguna. Fungsi `selectFlower(index)` akan menyimpan index bunga ke dalam StateFlow dan mencatat log di Logcat. Fungsi ini dipanggil setiap kali pengguna menekan tombol “Detail”. Karena ViewModel tidak bergantung pada UI secara langsung, maka jika layar diputar atau UI di-recompose, data bunga tetap tersimpan dan tidak hilang. Ini adalah keunggulan utama **penggunaan ViewModel**.

5. BirthFlowerViewModelFactory.kt

File ini untuk memberikan instance `BirthFlowerViewModel` ke dalam Compose UI. Ini juga membantu memisahkan pembuatan ViewModel dari UI, yang membuat kode lebih bersih dan modular.

2. Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

JAWAB: Application class adalah kelas yang dijalankan pertama kali saat aplikasi Android dibuka, sebelum Activity, Service, atau komponen lainnya.

Fungsi utamanya:

- Menyediakan tempat untuk inisialisasi global, seperti database, library, atau konfigurasi awal.
- Digunakan untuk menyimpan data global yang bisa diakses dari seluruh bagian aplikasi.
- Cocok untuk mengatur logika yang bersifat umum, misalnya logging atau pengaturan bahasa aplikasi.

MODUL 5 : Connect to the Internet

SOAL 1

Soal Praktikum:

1. Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a) Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- b) Gunakan KotlinX Serialization sebagai library JSON.
- c) Gunakan library seperti Coil atau Glide untuk image loading.
- d) API yang digunakan pada modul ini bebas, contoh API gratis The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: <https://developer.themoviedb.org/docs/getting-started>
- e) Implementasikan konsep data persistence (misalnya offline-first app, pengaturan dark/light mode, fitur favorite, dll)
- f) Gunakan caching strategy pada Room..
- g) Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

1. FlowerRemote.kt

Tabel 15. Source Code Soal 1 Modul 5

1	package com.example.modul5.model
2	
3	import kotlinx.serialization.SerialName
4	import kotlinx.serialization.Serializable
5	
6	@Serializable
7	data class FlowerRemote(
8	@SerialName("name") val name: String,
9	@SerialName("symbol") val symbol: String,
10	@SerialName("wikiLink") val wikiLink: String,
11	@SerialName("imageUrl") val imageUrl: String
12	)

2. FlowerApiService.kt

Tabel 16. Source Code Soal 1 Modul 5

1	package com.example.modul5.network
2	
3	import com.example.modul5.model.FlowerRemote
4	import retrofit2.http.GET
5	
6	interface FlowerApiService {
7	@GET("flowers")
8	suspend fun getFlowers(): List<FlowerRemote>
9	}

3. ApiClient.kt

Tabel 17. Source Code Soal 1 Modul 5

1	package com.example.modul5.network
2	
3	import
4	com.jakewharton.retrofit2.converter.kotlinx.serialization.
5	asConverterFactory
6	import kotlinx.serialization.json.Json
7	import okhttp3.MediaType.Companion.toMediaType
8	import retrofit2.Retrofit
9	
10	
11	@OptIn(kotlinx.serialization.ExperimentalSerializati
12	onApi::class)
13	object ApiClient {
14	private const val BASE_URL =
15	"https://birthflowersapi.free.beeceptor.com/"
16	
17	private val json = Json {
18	ignoreUnknownKeys = true
19	}
20	
21	private val retrofit = Retrofit.Builder()
22	.baseUrl(BASE_URL)
23	
24	.addConverterFactory(json.asConverterFactory("applic
25	ation/
26	json".toMediaType()))
27	.build()

28	
29	val apiService: FlowerApiService =
30	retrofit.create(FlowerApiService::class.java)
	}

4. BirthFlowerViewModel.kt

Tabel 18. Source Code Soal 1 Modul 5

1	package com.example.modul5
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.viewModelScope
5	import com.example.modul5.model.FlowerRemote
6	import com.example.modul5.network.ApiClient
7	import kotlinx.coroutines.flow.MutableStateFlow
8	import kotlinx.coroutines.flow.StateFlow
9	import kotlinx.coroutines.launch
10	import android.util.Log
11	
12	class BirthFlowerViewModel : ViewModel() {
13	
14	private val _flowers =
15	MutableStateFlow<List<FlowerRemote>>(emptyList())
16	val flowers: StateFlow<List<FlowerRemote>> =
17	_flowers
18	
19	init {
20	fetchFlowers()
21	}

22	
23	private fun fetchFlowers() {
24	viewModelScope.launch {
25	try {
26	val response =
27	ApiClient.apiService.getFlowers()
28	_flowers.value = response
29	Log.d("BirthFlowerViewModel",
30	"Loaded \${response.size} flowers")
31	} catch (e: Exception) {
32	Log.e("BirthFlowerViewModel", "API
33	Error", e)
34	}
35	}
36	}
37	}

5. FlowerListScreen.kt

Tabel 19. Source Code Soal 1 Modul 5

1	package com.example.modul5.ui
2	
3	import android.content.Intent
4	import android.net.Uri
5	import androidx.compose.foundation.layout.*
6	import androidx.compose.foundation.lazy.itemsIndexed
7	import androidx.compose.foundation.lazy.LazyColumn
8	import
9	androidx.compose.foundation.shape.RoundedCornerShape
10	import androidx.compose.material3.*

11	import androidx.compose.runtime.Composable
12	import androidx.compose.runtime.collectAsState
13	import androidx.compose.ui.Alignment
14	import androidx.compose.ui.Modifier
15	import androidx.compose.ui.platform.LocalContext
16	import androidx.compose.ui.text.font.FontWeight
17	import androidx.compose.ui.unit.dp
18	import androidx.compose.ui.unit.sp
19	import
20	androidx.lifecycle.viewmodel.compose.viewModel
21	import androidx.navigation.NavController
22	import coil.compose.AsyncImage
23	import coil.request.ImageRequest
24	import com.example.modul5.model.FlowerRemote
25	import com.example.modul5.BirthFlowerViewModel
26	
27	@Composable
28	fun FlowerListScreen(navController: NavController) {
29	val viewModel: BirthFlowerViewModel =
30	viewModel()
31	val flowers = viewModel.flowers.collectAsState()
32	val flowerList = flowers.value
33	
34	LazyColumn(modifier = Modifier.padding(16.dp)) {
35	itemsIndexed(flowerList) { index: Int,
36	flower: FlowerRemote ->
37	FlowerItem(
38	flower = flower,
39	index = index,
40	navController = navController

41	)
42	}
43	}
44	}
45	
46	@Composable
47	fun FlowerItem(
48	flower: FlowerRemote,
49	index: Int,
50	navController: NavController
51	) {
52	val context = LocalContext.current
53	
54	Card(
55	shape = RoundedCornerShape(8.dp),
56	modifier = Modifier
57	.fillMaxWidth()
58	.padding(8.dp)
59	) {
60	Row(
61	modifier = Modifier
62	.padding(16.dp),
63	verticalAlignment =
64	Alignment.CenterVertically
65	) {
66	AsyncImage(
67	model =
68	ImageRequest.Builder(context)
69	.data(flower.imageUrl)
70	.crossfade(true)

71	.build(),
72	contentDescription = flower.name,
73	modifier = Modifier.size(100.dp)
74	)
75	
76	Spacer(modifier = Modifier.width(16.dp))
77	
78	Column(modifier = Modifier.weight(1f)) {
79	Text(
80	text = flower.name,
81	style =
82	MaterialTheme.typography.headlineSmall.copy(
83	fontWeight = FontWeight.Bold
84	)
85	)
86	Spacer(modifier =
87	Modifier.height(4.dp))
88	Text(
89	text = flower.symbol,
90	style =
91	MaterialTheme.typography.bodyMedium
92	)
93	Spacer(modifier =
94	Modifier.height(8.dp))
95	
96	Row(
97	horizontalArrangement =
98	Arrangement.spacedBy(8.dp)
99	) {
100	Button(onClick = {

101	val intent =
102	Intent(Intent.ACTION_VIEW,
103	Uri.parse(flower.wikiLink))
104	
105	context.startActivity(intent)
106	}) {
107	Text(text = "Wiki", fontSize
108	= 12.sp)
109	}
110	
111	Button(onClick = {
112	
113	navController.navigate("detail/\$index")
114	}) {
116	Text(text = "Detail",
117	fontSize = 12.sp)
118	}
119	}
120	}
121	}
122	}
123	}
124	

6. FlowerDetailScreen.kt

Tabel 20. Source Code Soal 1 Modul 5

1	package com.example.modul5.ui
2	
3	import androidx.compose.foundation.layout.*

4	import androidx.compose.material3.*
5	import androidx.compose.runtime.Composable
6	import androidx.compose.ui.Modifier
7	import androidx.compose.ui.platform.LocalContext
8	import androidx.compose.ui.text.font.FontWeight
9	import androidx.compose.ui.unit.dp
10	import androidx.compose.ui.unit.sp
11	import coil.compose.AsyncImage
12	import coil.request.ImageRequest
13	import com.example.modul5.model.FlowerRemote
14	import androidx.compose.ui.Alignment
15	import androidx.compose.ui.text.style.TextAlign
16	
17	@Composable
18	fun FlowerDetailScreen(flower: FlowerRemote) {
19	val context = LocalContext.current
20	
21	Column(
22	modifier = Modifier
23	.padding(24.dp)
24	.fillMaxSize(),
25	verticalArrangement = Arrangement.Top,
26	horizontalAlignment =
27	Alignment.CenterHorizontally
28	) {
29	AsyncImage(
30	model = ImageRequest.Builder(context)
31	.data(flower.imageUrl)
32	.crossfade(true)
33	.build(),

34	contentDescription = flower.name,
35	modifier = Modifier.size(280.dp)
36	)
37	
38	Spacer(modifier = Modifier.height(24.dp))
39	
40	Text(
41	text = flower.name,
42	fontSize = 26.sp,
43	fontWeight = FontWeight.Bold
44	)
45	
46	Spacer(modifier = Modifier.height(12.dp))
47	
48	Text(
49	text = flower.symbol,
50	style =
51	MaterialTheme.typography.bodyLarge,
52	textAlign = TextAlign.Center
53	)
54	}
	}

7. MainActivity.kt

Tabel 21. Source Code Soal 1 Modul 5

1	package com.example.modul5
2	
3	import android.os.Bundle

4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.compose.foundation.layout.padding
7	import androidx.compose.material.icons.Icons
8	import
9	androidx.compose.material.icons.filled.DarkMode
10	import
11	androidx.compose.material.icons.filled.LightMode
12	import androidx.compose.material3.*
13	import androidx.compose.runtime.collectAsState
14	import androidx.compose.ui.Modifier
15	import
16	androidx.lifecycle.viewmodel.compose.viewModel
17	import androidx.navigation.compose.NavHost
18	import androidx.navigation.compose.composable
19	import
20	androidx.navigation.compose.rememberNavController
21	import com.example.modul5.ui.FlowerDetailScreen
22	import com.example.modul5.ui.FlowerListScreen
23	import
24	com.example.modul5.ui.theme.BirthFlowerzAppTheme
25	
26	@OptIn(ExperimentalMaterial3Api::class)
27	class MainActivity : ComponentActivity() {
28	override fun onCreate(savedInstanceState:
29	Bundle?) {
30	super.onCreate(savedInstanceState)
31	setContent {
32	val themeViewModel: ThemeViewModel =
33	viewModel()

```

34         val isDark =
35 themeViewModel.isDarkMode.collectAsState()
36
37         BirthFlowerzAppTheme(darkTheme =
38 isDark.value) {
39             val navController =
40 rememberNavController()
41
42             Scaffold(
43                 topBar = {
44                     TopAppBar(
45                         title = { Text("Birth
46 Flowers") },
47                         actions = {
48                             IconButton(onClick =
49 { themeViewModel.toggleTheme() }) {
50                                 Icon(
51                                     imageVector
52 = if (isDark.value) Icons.Filled.DarkMode else
53 Icons.Filled.LightMode,
54
55 contentDescription = "Toggle Theme"
56                                     )
57                                 }
58                             }
59                         )
60                     }
61                 ) { innerPadding ->
62                     NavHost(
63                         navController =

```

```

64 navController,
65             startDestination =
66 "main_screen",
67             modifier =
68 Modifier.padding(innerPadding)
69         ) {
70             composable("main_screen") {
71
72 FlowerListScreen(navController = navController)
73             }
74
75 composable("detail/{flowerIndex}") { backStackEntry
76 ->
77             val index =
78 backStackEntry.arguments?.getString("flowerIndex")?.
79 toIntOrNull()
80             val flowerViewModel:
81 BirthFlowerViewModel = viewModel()
82             val flowers =
83 flowerViewModel.flowers.collectAsState().value
84             val flower = index?.let
85 { flowers.getOrNull(it) }
86             flower?.let {
87
88 FlowerDetailScreen(flower = it)
89             }
90         }
91     }
92 }
93 }

```

94	}
95	}
96	}

8. ThemeViewModel.kt

Tabel 22. Source Code Soal 1 Modul 5

1	package com.example.modul5
2	
3	import android.app.Application
4	import androidx.lifecycle.AndroidViewModel
5	import androidx.lifecycle.viewModelScope
6	import com.example.modul5.datastore.ThemePreference
7	import kotlinx.coroutines.flow.MutableStateFlow
8	import kotlinx.coroutines.flow.StateFlow
9	import kotlinx.coroutines.flow.first
10	import kotlinx.coroutines.launch
11	
12	class ThemeViewModel(application: Application) :
13	AndroidViewModel(application) {
14	
15	private val _isDarkMode =
16	MutableStateFlow(false)
17	val isDarkMode: StateFlow<Boolean> get() =
18	_isDarkMode
19	
20	init {
21	viewModelScope.launch {
22	val current =

23	ThemePreference.getThemeSetting(getApplication()).fi
24	rst()
25	_isDarkMode.value = current
26	}
27	}
28	
29	fun toggleTheme() {
30	val context = getApplication<Application>()
31	viewModelScope.launch {
32	val newValue = !_isDarkMode.value
33	
34	ThemePreference.saveThemeSetting(context, newValue)
35	_isDarkMode.value = newValue
36	}
37	}
38	}

9. ThemePreference.kt

Tabel 23. Source Code Soal 1 Modul 5

1	package com.example.modul5.datastore
2	
3	import android.content.Context
4	import
5	androidx.datastore.preferences.core.booleanPreferen
6	cesKey
7	import androidx.datastore.preferences.core.edit
8	import

9	<code>androidx.datastore.preferences.preferencesDataStore</code>
10	<code>import kotlinx.coroutines.flow.Flow</code>
11	<code>import kotlinx.coroutines.flow.map</code>
12	
13	<code>private const val DATASTORE_NAME = "settings"</code>
14	<code>val Context.dataStore by preferencesDataStore(name</code>
15	<code>= DATASTORE_NAME)</code>
16	
17	<code>object ThemePreference {</code>
18	<code> private val DARK_MODE_KEY =</code>
19	<code>booleanPreferencesKey("dark_mode")</code>
20	
21	<code> fun getThemeSetting(context: Context):</code>
22	<code>Flow<Boolean> {</code>
23	<code> return context.dataStore.data.map {</code>
24	<code>preferences -></code>
25	<code> preferences[DARK_MODE_KEY] ?: false</code>
26	<code> }</code>
27	<code> }</code>
28	
29	<code> suspend fun saveThemeSetting(context: Context,</code>
30	<code>isDarkMode: Boolean) {</code>
31	<code> context.dataStore.edit { preferences -></code>
32	<code> preferences[DARK_MODE_KEY] = isDarkMode</code>
33	<code> }</code>
34	<code> }</code>
35	<code>}</code>

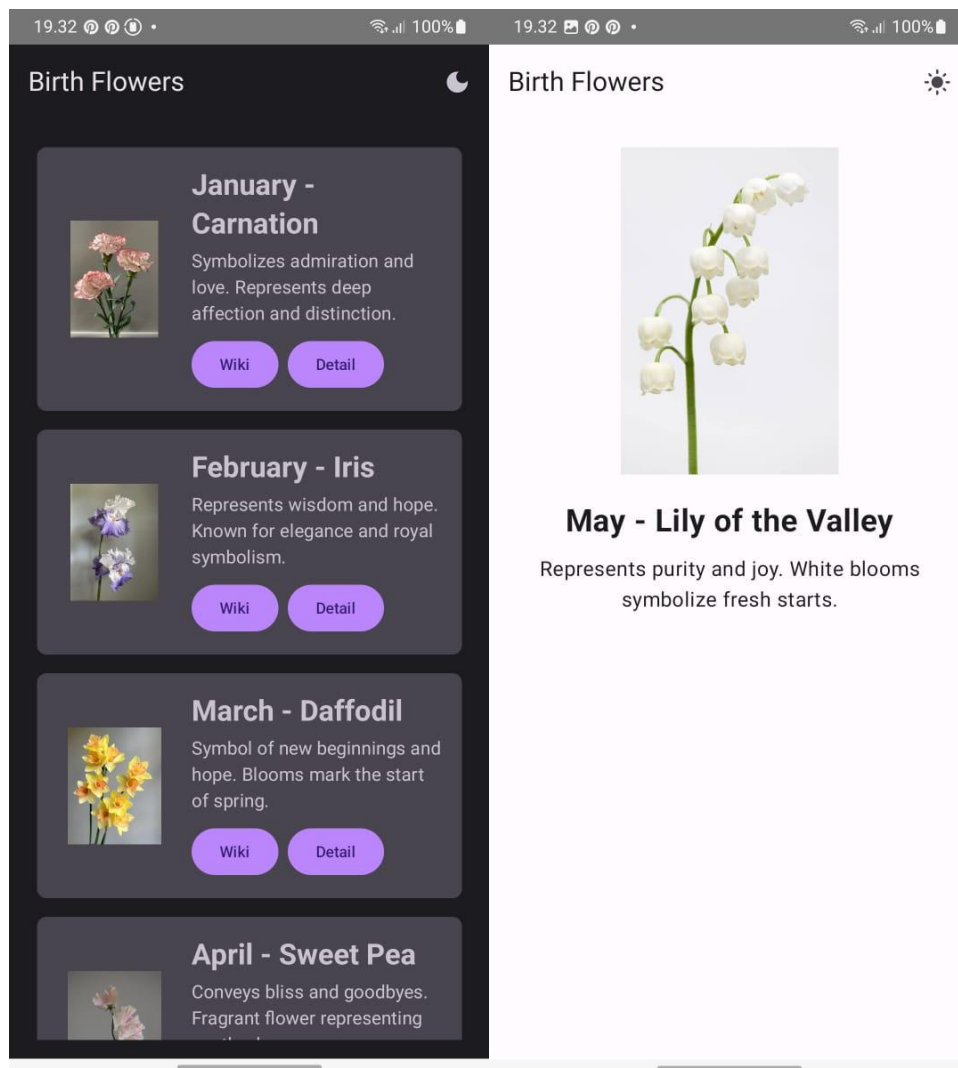
10. Theme.kt

Tabel 24. Source Code Soal 1 Modul 5

1	package com.example.modul5.ui.theme
2	
3	import androidx.compose.material3.MaterialTheme
4	import androidx.compose.material3.darkColorScheme
5	import androidx.compose.material3.lightColorScheme
6	import androidx.compose.runtime.Composable
7	import androidx.compose.material3Typography
8	import androidx.compose.material3.ColorScheme
9	
10	private val <i>DarkColorScheme</i> = <i>darkColorScheme</i> (
11	primary =
12	androidx.compose.ui.graphics.Color(0xFFBB86FC),
13	secondary =
14	androidx.compose.ui.graphics.Color(0xFF03DAC6)
15	)
16	
17	private val <i>LightColorScheme</i> = <i>lightColorScheme</i> (
18	primary =
19	androidx.compose.ui.graphics.Color(0xFF6200EE),
20	secondary =
21	androidx.compose.ui.graphics.Color(0xFF03DAC6)
22	)
23	
24	@Composable
25	fun BirthFlowerzAppTheme(
26	darkTheme: Boolean = false,
27	content: @Composable () -> Unit

```
28 ) {  
29     val colors: ColorScheme = if (darkTheme)  
30     DarkColorScheme else LightColorScheme  
31  
32     MaterialTheme(  
33         colorScheme = colors,  
34         typography = Typography(),  
35         content = content  
36     )  
37 }
```

B. Output Program



Gambar 7. Screenshot Hasil Jawaban Soal 1 Modul 5

C. Pembahasan

1. FlowerRemote.kt

File ini berisi *data class* yang digunakan untuk menampung data dari API.

Anotasi `@Serializable` dan `@SerializedName` digunakan dari library `kotlinx.serialization` agar JSON dari server bisa langsung dipetakan ke objek Kotlin.

- `name`: nama bunga
- `symbol`: makna atau simbolis dari bunga tersebut
- `wikiLink`: link ke Wikipedia untuk informasi lengkap
- `imageUrl`: alamat gambar bunga

File ini penting karena menjadi jembatan antara data dari internet dan tampilan aplikasi.

2. FlowerApiService.kt

File ini adalah interface Retrofit yang mendefinisikan endpoint API.

Fungsi `getFlowers()` menggunakan anotasi `@GET("flowers")` untuk mengambil data dari URL `https://birthflowersapi.free.beeceptor.com/flowers`.

Kata `suspend` menunjukkan bahwa fungsi ini berjalan secara asynchronous dengan coroutine. Hasilnya berupa list dari `FlowerRemote`, yaitu data bunga yang akan ditampilkan ke layar. Singkatnya, file ini mendefinisikan *apa* yang mau diambil dari internet.

3. ApiClient.kt

File ini berfungsi untuk menyiapkan Retrofit, yaitu library yang digunakan untuk komunikasi internet.

- `BASE_URL` adalah alamat dasar dari API.
- `Json { ignoreUnknownKeys = true }` memastikan kalau JSON yang tidak dikenali tidak bikin error.
- `asConverterFactory(...)` memungkinkan penggunaan `kotlinx.serialization` sebagai converter JSON.
- `retrofit.create(FlowerApiService::class.java)` membuat objek `apiService` yang nantinya bisa dipakai di `ViewModel`.

File ini hanya dibuat sekali dan dipakai di mana saja saat ingin ambil data dari internet.

4. BirthFlowerViewModel.kt

File ini berfungsi sebagai pengelola data utama aplikasi. `BirthFlowerViewModel` adalah kelas turunan dari `ViewModel` yang menggunakan `StateFlow` untuk menyimpan dan mengalirkan data daftar bunga secara reaktif. Di dalamnya, fungsi `fetchFlowers()` dipanggil saat `ViewModel` pertama kali dibuat melalui `init`, dan digunakan untuk mengambil data dari API menggunakan `Retrofit` yang sudah disiapkan di `ApiClient`. Fungsi ini berjalan dalam `viewModelScope.launch`, artinya berjalan secara asynchronous agar tidak membebani UI. Jika sukses, datanya dimasukkan ke `_flowers`, dan bila gagal akan muncul log error di `Logcat`. Pendekatan ini juga memastikan data tetap aman saat orientasi layar berubah, karena `ViewModel` tidak ikut dihancurkan seperti `Activity`.

5. FlowerListScreen.kt

File ini menampilkan daftar bunga dalam bentuk scrollable list menggunakan `LazyColumn` dari `Jetpack Compose`. Data bunga diambil dari `BirthFlowerViewModel` dengan `collectAsState()` sehingga tampilannya akan otomatis ter-update saat data berubah. Di dalam list, setiap item ditampilkan

menggunakan komponen `FlowerItem`. Komponen ini terdiri dari gambar bunga, nama, dan simbolnya, serta dua tombol. Tombol pertama membuka halaman Wikipedia dari bunga tersebut menggunakan `Intent`, sedangkan tombol kedua akan mengarahkan pengguna ke halaman detail dengan mengirimkan index item ke `NavController`. File ini menerapkan prinsip pemisahan logika dan UI dengan baik, karena semua logika data disimpan di `ViewModel`, sementara file ini hanya fokus pada tampilan dan interaksi pengguna.

6. FlowerDetailScreen.kt

File ini menampilkan halaman detail untuk satu bunga yang dipilih dari list. Komponen ini menerima parameter `flower` yang berisi data lengkap dari `FlowerRemote`, seperti nama, gambar, dan simbol. Layout-nya tersusun secara vertikal dengan `Column`, dan gambar bunga ditampilkan menggunakan `AsyncImage` dari `Coil`, yang langsung memuat gambar dari internet. Di bawah gambar, ditampilkan nama bunga dengan ukuran font besar dan tebal, lalu simbol bunga dengan gaya body text dan rata tengah. Tampilan ini sederhana namun efektif untuk memberikan informasi detail bunga kepada pengguna setelah mereka menekan tombol "Detail" dari halaman sebelumnya.

7. MainActivity.kt

File `MainActivity.kt` adalah titik masuk utama aplikasi saat dijalankan. Di dalam metode `onCreate()`, aplikasi menggunakan `setContent` untuk menampilkan UI berbasis Jetpack Compose. Hal pertama yang dilakukan adalah mengambil instance dari `ThemeViewModel`, lalu membaca nilai `isDarkMode` yang menunjukkan apakah tema saat ini gelap atau terang. Komponen `BirthFlowerzAppTheme` kemudian digunakan untuk menerapkan tema tersebut ke seluruh aplikasi. Navigasi antar layar diatur menggunakan `NavHost`, dengan dua layar utama: `main_screen`

yang menampilkan daftar bunga, dan `detail/{flowerIndex}` yang menampilkan detail bunga berdasarkan indeks. Fitur dark mode dikontrol melalui `AppBar`, yang memiliki tombol toggle dengan ikon `DarkMode` dan `LightMode`. Saat pengguna menekan tombol tersebut, fungsi `toggleTheme()` akan dijalankan untuk mengubah tema secara real-time dan menyimpannya.

8. ThemeViewModel.kt

`ThemeViewModel.kt` adalah `ViewModel` khusus untuk mengelola preferensi tema aplikasi (gelap atau terang). Kelas ini mewarisi `AndroidViewModel` agar bisa mengakses konteks aplikasi secara langsung. Di dalamnya terdapat `MutableStateFlow` yang menyimpan status dark mode dan dapat diamati secara real-time oleh komponen UI. Saat `ViewModel` dibuat (`init`), ia langsung membaca nilai yang disimpan di `DataStore` menggunakan fungsi `getThemeSetting()`, lalu memperbarui nilai `_isDarkMode`. Fungsi `toggleTheme()` digunakan untuk membalikkan nilai tema saat ini, menyimpannya kembali ke `DataStore`, dan memperbarui nilai yang diamati oleh UI. Dengan pendekatan ini, perubahan tema dapat dipertahankan bahkan setelah aplikasi ditutup dan dibuka kembali.

9. ThemePreference.kt

File ini berisi objek `ThemePreference`, yang bertanggung jawab menyimpan dan mengambil pengaturan dark mode menggunakan `Jetpack DataStore`. Data disimpan dalam bentuk boolean di dalam file `settings.preferences_pb`. Dengan menggunakan key `DARK_MODE_KEY`, fungsi `getThemeSetting()` mengembalikan `Flow<Boolean>` yang terus memantau perubahan preferensi, sementara fungsi `saveThemeSetting()` digunakan untuk menyimpan nilai baru ke `DataStore`. Karena operasi ini berjalan asynchronous, `suspend` digunakan agar

bisa dipanggil dari coroutine. `Context.dataStore` juga dibuat sebagai ekstensi agar mudah diakses dari kelas mana pun yang memiliki konteks Android.

10. Theme.kt

File ini mendefinisikan dua skema warna utama aplikasi: satu untuk mode terang dan satu lagi untuk mode gelap. Warna-warna ini ditentukan menggunakan `darkColorScheme` dan `lightColorScheme` dari Material 3. Fungsi `BirthFlowerzAppTheme()` menerima parameter `darkTheme` untuk memilih skema warna yang akan digunakan, lalu menerapkan skema dan tipografi melalui `MaterialTheme`. Fungsi ini membungkus seluruh UI aplikasi, memastikan semua komponen mengikuti gaya warna yang konsisten. Dengan pendekatan ini, pengguna dapat melihat perubahan warna langsung saat beralih dari light ke dark mode atau sebaliknya, dengan pengalaman visual yang mulus.

Tautan Git

Berikut adalah tautan untuk semua source code modul pemrograman mobile.

<https://github.com/rgnsilvas/Pemrograman-Mobile/tree/main>